



Performance

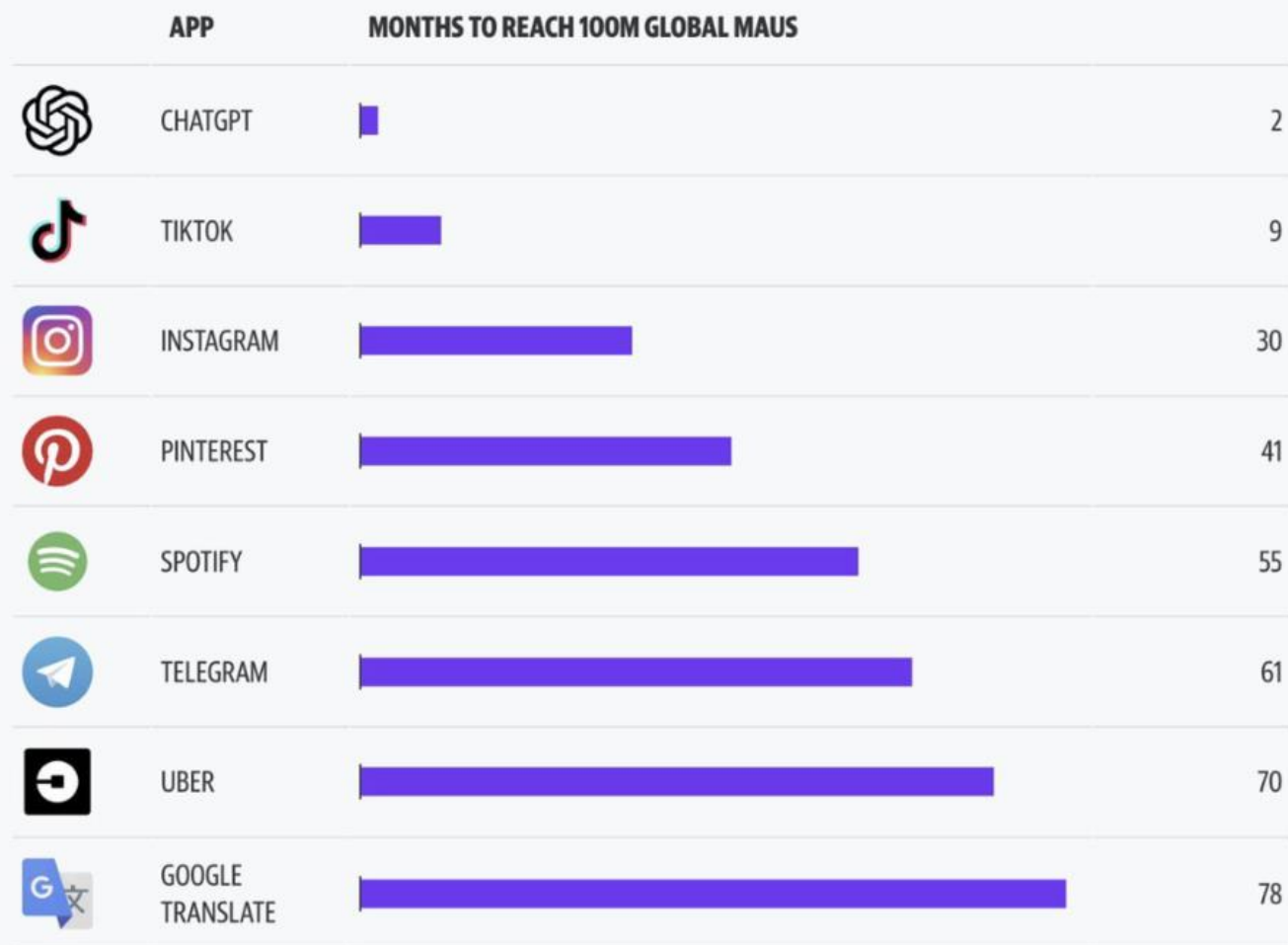
- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot

Number of years it took for each product to gain **50 MILLION** users:

Airlines	Automobiles	Telephone	Electricity	Credit Card	Television	ATMs	Computer	Cell Phone
								
68 years	62 years	50 years	46 years	28 years	22 years	18 years	14 years	12 years
Debit Cards	Internet	PayPal	iPods	YouTube	Facebook	Twitter	Instagram	Pokémon Go
								
12 years	7 years	5 years	4 years	4 years	3 years	2 years	17 months	19 days

HOW LONG IT TOOK TOP APPS TO HIT 100M MONTHLY USERS

ChatGPT is estimated to have hit 100M users in January, 2 months after it's launch.
Here's how long it took other top apps to reach that:



SOURCE: UBS

Performance desde los atributos

- Es un **atributo de calidad operacional**, observable cuando el sistema está en ejecución
- **Carga de trabajo** que una aplicación debe ejecutar **en una unidad de tiempo, y/o *deadlines* que debe cumplir** para una correcta operación.
- Capacidad del software para **dar una respuesta y ejecutar sus tareas**
 - Depende de la comunicación e interacción entre componentes del sistema

Tácticas de mejora de desempeño (Bass, Kazman)

Manejo de recursos

- Incrementar recursos (vertical)
- Introducir concurrencia (horizontal micro)
- Múltiples copias de computación (horizontal macro)
- Múltiples copias de datos (Cachés)
- Uso de colas
- Manejo de prioridad en recursos
- Uso de infraestructura elástica (*cloud* en algunos casos)



Tácticas de mejora de desempeño

Control de demanda

- Limitar respuestas (429: Too many requests)
- Priorizar eventos en arquitecturas distribuidas
- Reducir *overhead* y limpiar basura
- Limitar tiempos de ejecución (*Timeouts*)
- Incrementar eficiencia de uso de recursos, aproximaciones



429 Too Many Requests

You have sent too many requests in a given amount of time.

Estrategias de optimización

- Determinación de responsabilidades
 - Permite apuntar responsables
- Modelos de coordinación
 - Timings y connascence
- Interdependencias
 - Connascence y acoplamiento
- Elección del stack
 - Tecnologías pueden apuntar a problemas distintos
 - Lenguaje: Go, Rust, Node, Scala
 - Framework: NestJS, Spring, RoR





Métricas clave de fitness en el performance

- *Throughput*
 - Peticiones / tiempo
 - Respuestas / tiempo
 - Errores / tiempo
- *Bandwidth*
- Latencia (ms - s)
 - Jitter
- Capacidad de conexiones
- Utilización de recursos
 - Accesos a bases de datos
 - *Uptime*
 - *Garbage Collector*
- Escalabilidad



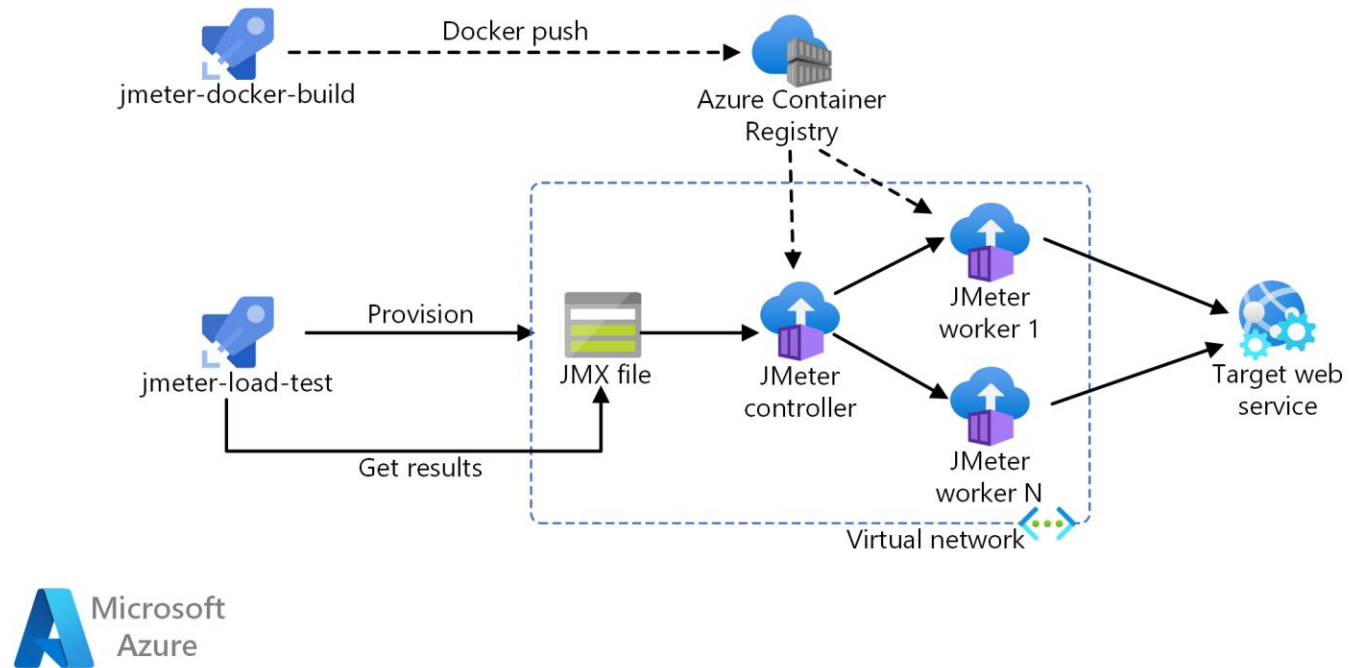
Como medir performance

- Experimental
 - Herramientas
 - Ensayo y error
 - Medir implementaciones
 - *Throughput*
 - *Latency*
 - Nivel de servicio
- Teórico
 - Teoría de colas
 - Tasas de llegada
 - Simulaciones



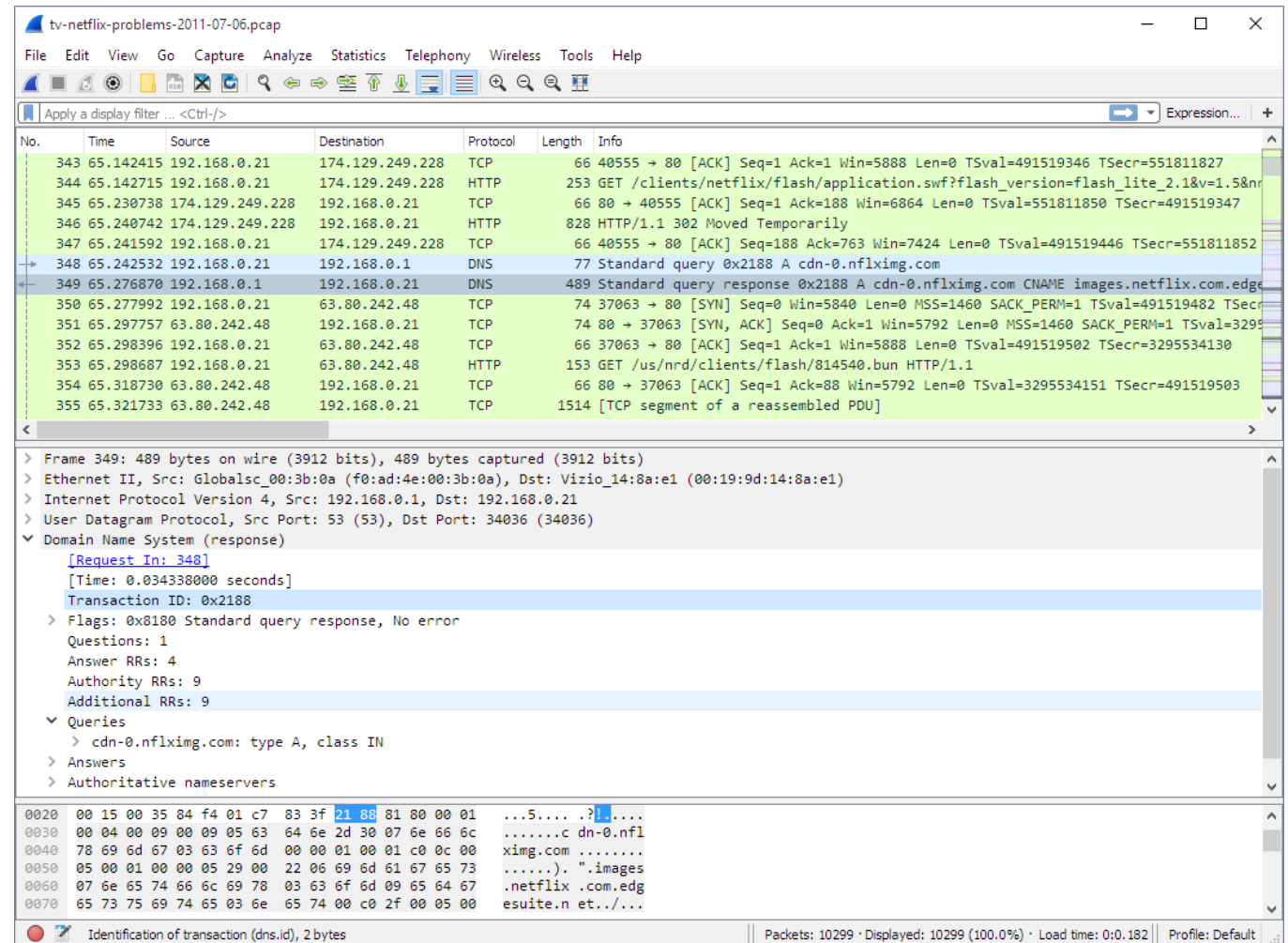
Experimental: Herramientas

- Herramientas de carga
 - Jmeter
 - Locust
 - K6
 - Vegeta
 - AWS distributed load testing
- Herramientas de monitoreo
 - Proveedores de monitoreo
 - New Relic (E3)
 - Datadog
 - Prometheus



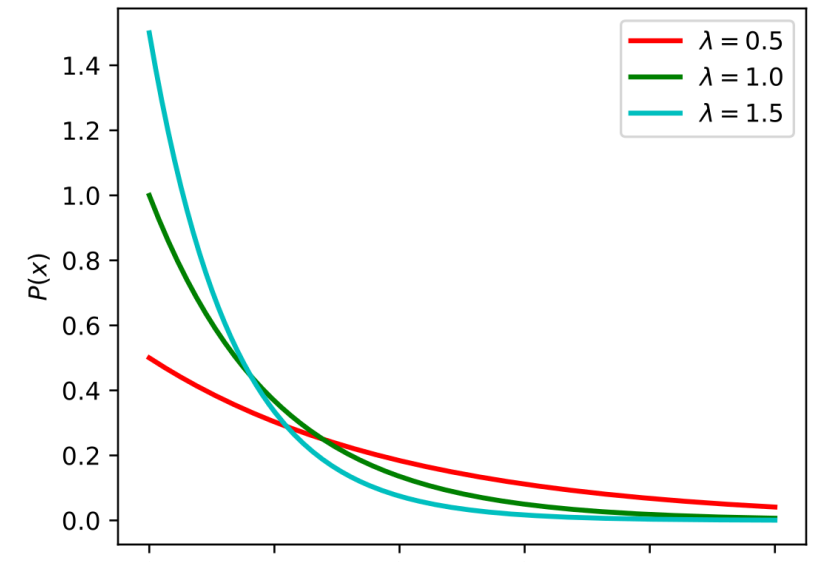
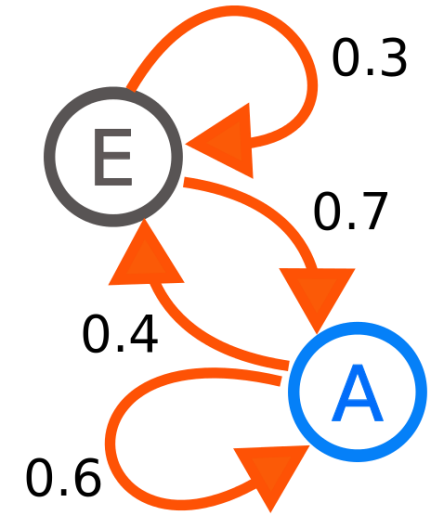
Experimental: Herramientas

- Herramientas de red
 - Browser
 - Wireshark
 - Ping
 - Iperf
- Herramientas de sistema
 - Htop/top
 - Time
 - Monitores de proceso de sistema



Teórico

- Teoría de colas
 - Análisis estadístico (llegada distribuye exponencial, Poisson)
- Cadenas de Markov
 - Análisis de capacidad y cuellos de botella
 - Identificar
 - Estados más probables del sistema (vivo, apenas, caído)
 - Transiciones más probables
 - Es un poco complejo modelar ese comportamiento



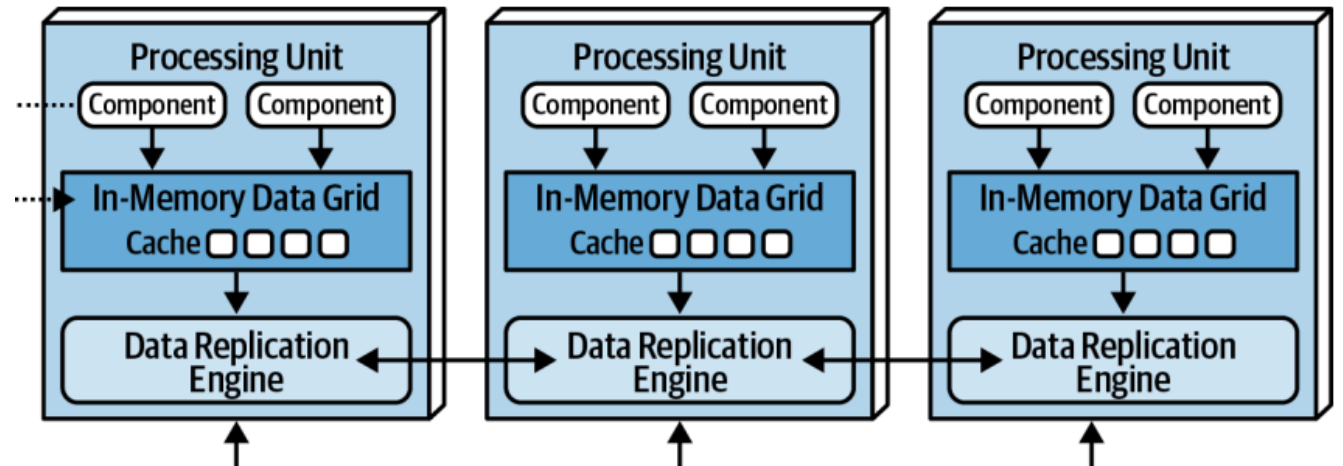


Factores base que afectan el performance

- Estilos
- Hardware
- Red y protocolos de comunicación
- Algoritmos y Estructuras de datos
- Priorizaciones
- Caching

Estilos, patrones y performance

- Algunos estilos tienen mejor performance que otros en escenarios distintos
- Estilos distribuidos pueden alcanzar altos niveles de performance
 - A cambio de consistencia, complejidad
 - E.g. SOA, Space-based, uS
- Ciertos estilos pueden penalizar performance
 - Capas
 - Enterprise SOA



Tamaño de la instancia	CPU virtual	Memoria (GiB)	Almacenamiento de instancias (GB)	Ancho de banda de red (Gbps)	Banda ancha de EBS (Gbps)
m6i.large	2	8	Solo EBS	Hasta 12,5	Hasta 10
m6i.xlarge	4	16	Solo EBS	Hasta 12,5	Hasta 10
m6i.2xlarge	8	32	Solo EBS	Hasta 12,5	Hasta 10
m6i.4xlarge	16	64	Solo EBS	Hasta 12,5	Hasta 10
m6i.8xlarge	32	128	Solo EBS	12,5	10
m6i.12xlarge	48	192	Solo EBS	18.75	15

Hardware y performance

- Impacto a nivel de máquina
- Su uso depende de la aplicación

La evolución del performance

- Impacto a nivel de máquina
- Diversos componentes afectan aspectos distintos
 - CPU
 - RAM
 - ASIC
 - HDD/SSD
- Ciertos componentes se aplican a contextos específicos
 - GPU
 - TPU
 - ASIC

DDR Generation	Data Rate (MT/s)	Command Rate (ns)	Bandwidth (GB/s)	Voltage (V)	Prefetch (bits)	Launch Year
DDR	200 - 400	10	1.6 - 3.2	2.5	2	2000
DDR2	400 - 1066	5	3.2 - 8.5	1.8	4	2003
DDR3	800 - 2133	2.5	6.4 - 17.0	1.5 (1.35 for DDR3L)	8	2007
DDR4	1600 - 6400	1.25	12.8 - 51.2	1.2 (1.05 for DDR4L)	8	2014
DDR5	4800 - 8400+	1.25	38.4 - 67.2+	1.1	16	2020

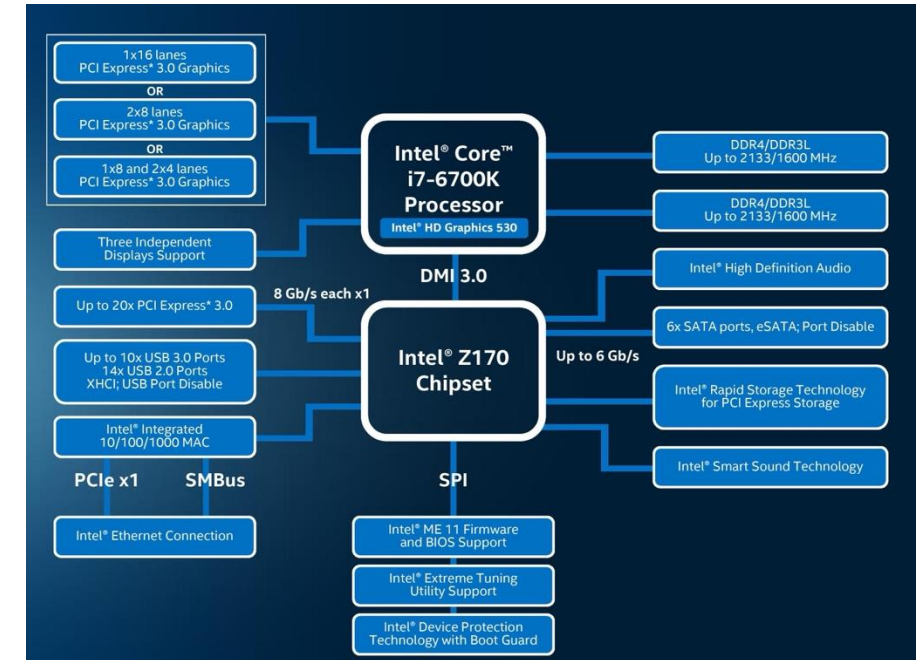


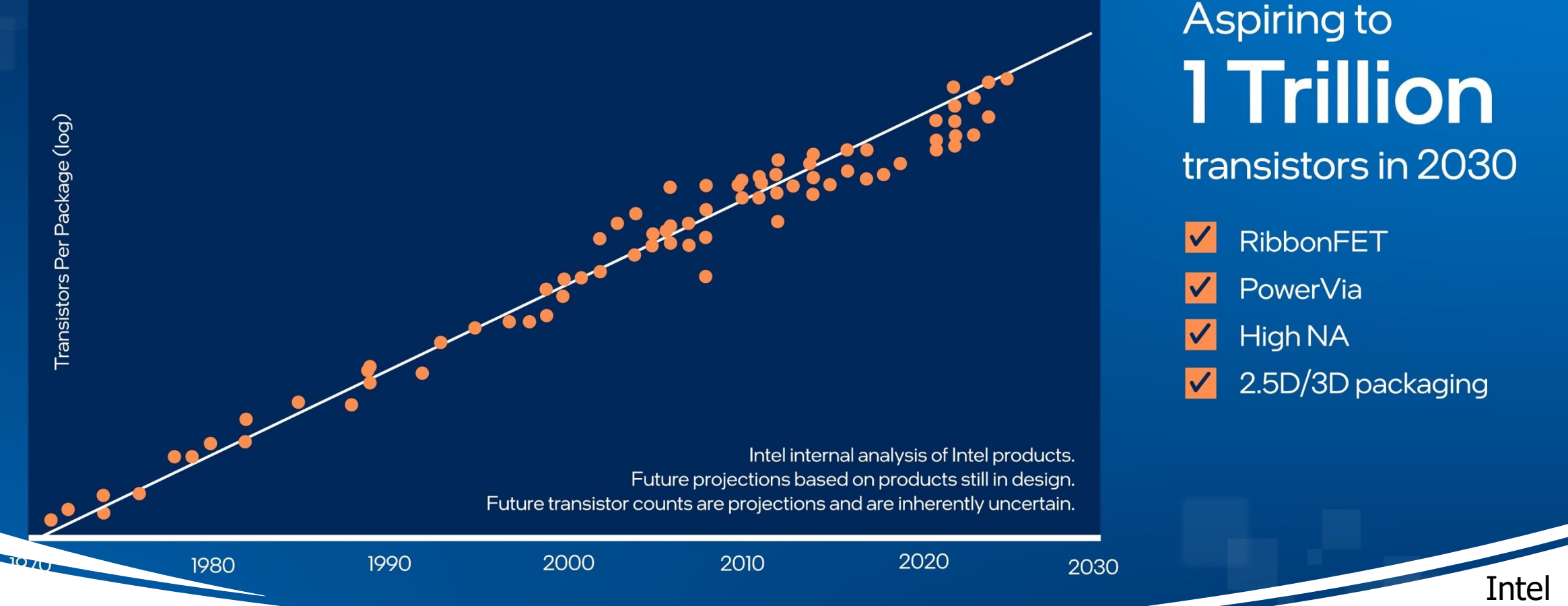
Performance II

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot

CPU

- Arquitectura del core influencia capacidad y rendimiento
 - Branch predictor
 - Instrucciones
 - Módulos específicos
- Cores y threads
 - Algunas aplicaciones se benefician de cores
 - Otras pueden paralelizarse para usar cores
- Clock
 - Actualmente en proceso de *capping*
 - Relación térmica limita escalamiento





La evolución del performance

- Constantemente escalando
- Procesadores de tipo específico mejoran

RAM

- Influye capacidad para mantener datos para procesar
 - Medido en GB/TB en la actualidad
- Generación
- Cantidad
- Latencia
- *Channels*



Tabla comparativa RAM

DDR Generation	Data Rate (MT/s)	Command Rate (ns)	Bandwidth (GB/s)	Voltage (V)	Prefetch (bits)	Launch Year
DDR	200 - 400	10	1.6 - 3.2	2.5	2	2000
DDR2	400 - 1066	5	3.2 - 8.5	1.8	4	2003
DDR3	800 - 2133	2.5	6.4 - 17.0	1.5 (1.35 for DDR3L)	8	2007
DDR4	1600 - 6400	1.25	12.8 - 51.2	1.2 (1.05 for DDR4L)	8	2014
DDR5	4800 - 8400+	1.25	38.4 - 67.2+	1.1	16	2020

Almacenamiento masivo

- Almacenamiento de datos de alta densidad, perdurable
- Más lento que RAM
- I/O bound
- Puede beneficiarse de ciertas arquitecturas (RAID, Hadoop)
- Arquitecturas

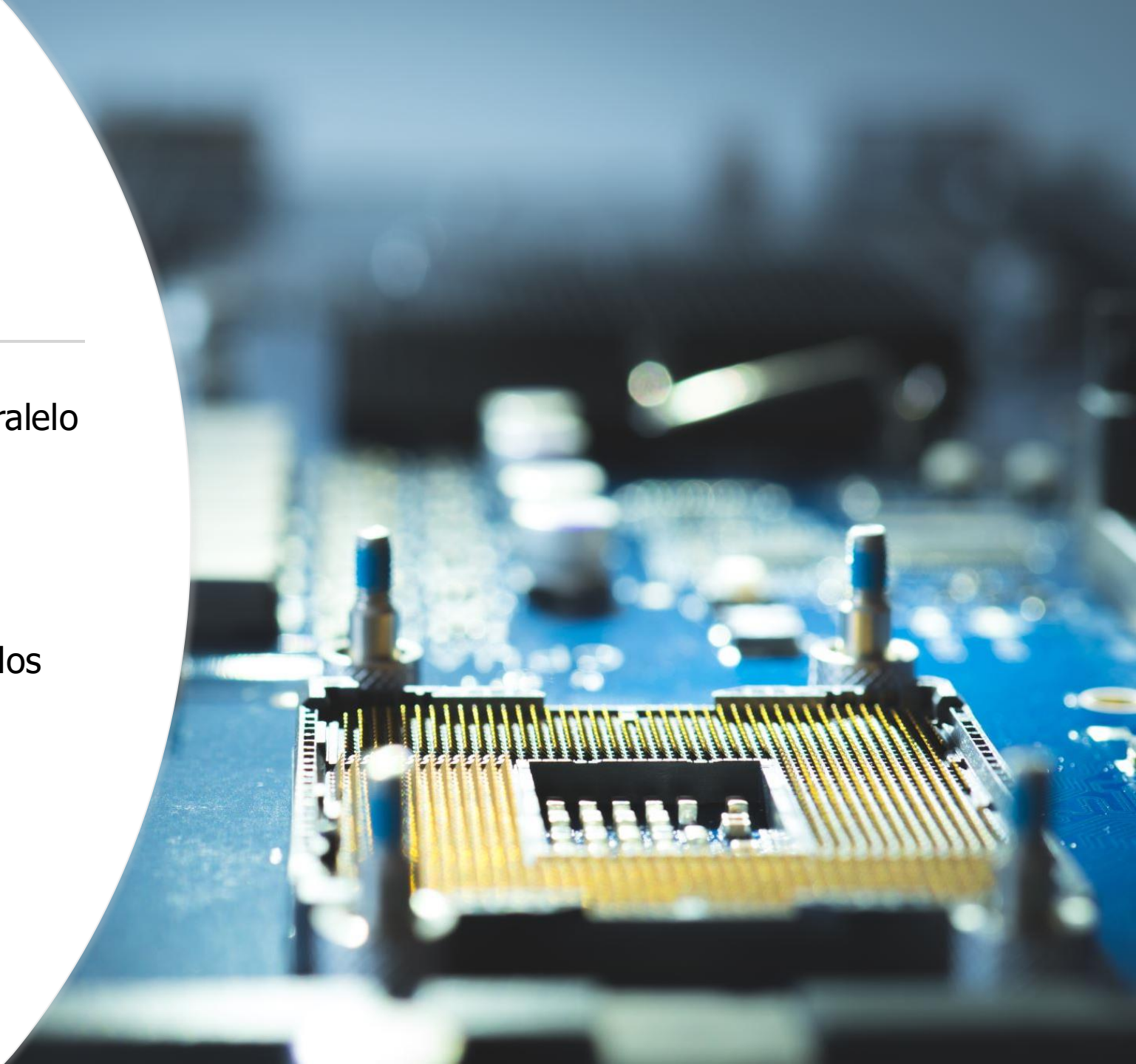


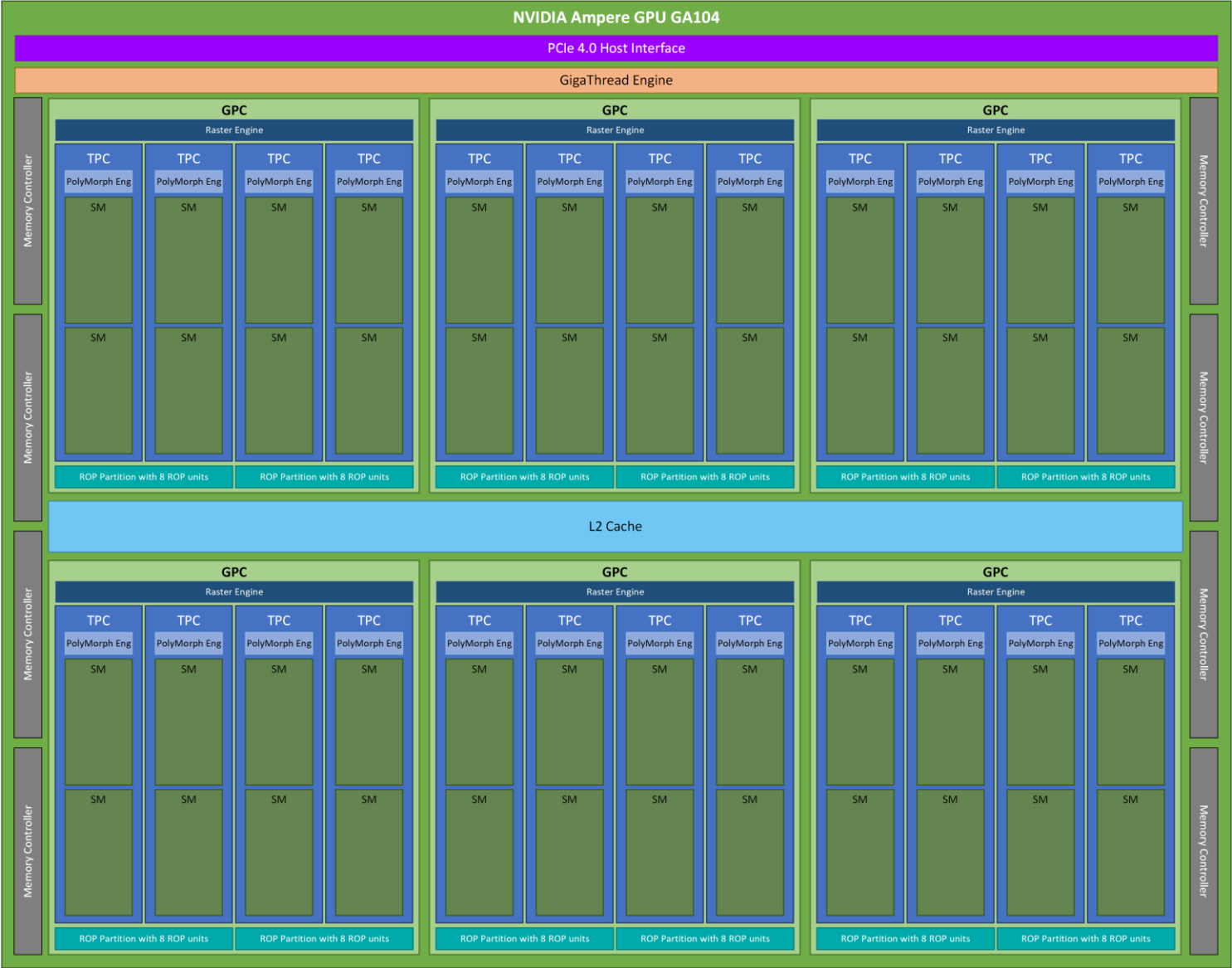
Tabla comparativa almacenamiento masivo

Tipo de almacenamiento	Capacidad	Velocidad	Costo	Durabilidad	Método de acceso	Casos de uso principales
Disco duro (HDD)	Hasta 20TB	50-150 MB/s	Bajo	Moderada	Aleatorio y secuencial	Almacenamiento general, copias de seguridad, archivos
Disco de estado sólido (SSD)	Hasta 8TB	200-3,500 MB/s	Moderado	Alta	Aleatorio y secuencial	Sistema operativo, aplicaciones, bases de datos
SSD NVMe	Hasta 8TB	1,000-7,000 MB/s	Alto	Alta	Aleatorio y secuencial	Aplicaciones de alto rendimiento, juegos, tareas intensivas en datos
HDD externo	Hasta 20TB	50-150 MB/s	Bajo a moderado	Moderada	Aleatorio y secuencial	Almacenamiento portátil, copias de seguridad, almacenamiento de medios
SSD externo	Hasta 4TB	200-1,000 MB/s	Moderado a alto	Alta	Aleatorio y secuencial	Almacenamiento portátil de alta velocidad, edición de medios, juegos
Memoria USB	Hasta 2TB	10-400 MB/s	Bajo a moderado	Moderada	Aleatorio y secuencial	Almacenamiento portátil, transferencia de datos, copias de seguridad
Tarjeta SD	Hasta 1TB	10-300 MB/s	Bajo a moderado	Moderada	Aleatorio y secuencial	Cámaras, teléfonos inteligentes, dispositivos portátiles
Disco óptico (CD/DVD/Blu-ray)	Hasta 128GB	1.32-72 MB/s	Bajo	Moderada a alta	Secuencial	Almacenamiento de medios, copias de seguridad, distribución de software

GPU

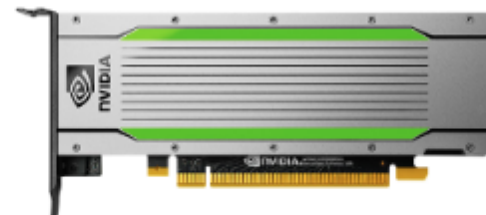
- Unidad específica para procesamiento paralelo masivo
 - Shaders
 - Pipelines
- Fuerte uso en aplicaciones de AI
- Actualmente aumentada por varios módulos
 - MMUs
 - Raytracing cores
 - Y otros
 - TPUs





TPU

- Unidad de procesamiento de tensores
- Ideal para operaciones de redes neuronales (vectoriales)
 - Entrenamiento
 - Inferencia
- Alta popularidad y costo (\$\$\$\$\$)
- Existen implementaciones mínimas experimentales

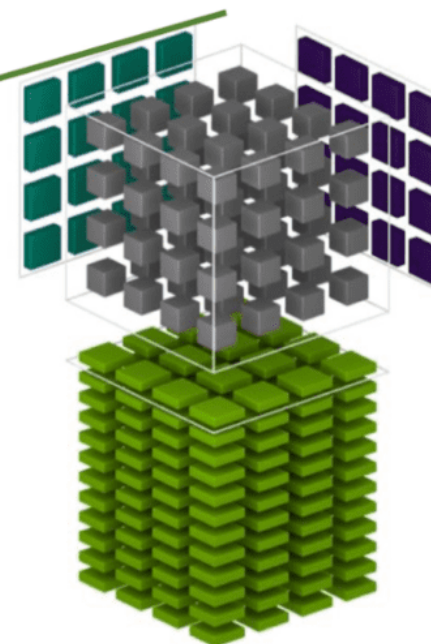


SPECIFICATIONS

GPU Architecture	NVIDIA Turing
NVIDIA Turing Tensor Cores	320
NVIDIA CUDA® Cores	2,560
Single-Precision	8.1 TFLOPS
Mixed-Precision (FP16/FP32)	65 TFLOPS
INT8	130 TOPS
INT4	260 TOPS
GPU Memory	16 GB GDDR6 300 GB/sec
ECC	Yes
Interconnect Bandwidth	32 GB/sec
System Interface	x16 PCIe Gen3
Form Factor	Low-Profile PCIe
Thermal Solution	Passive
Compute APIs	CUDA, NVIDIA TensorRT™, ONNX



```
y = net(x)
loss = lossfn(y, label)
loss.backward()
```



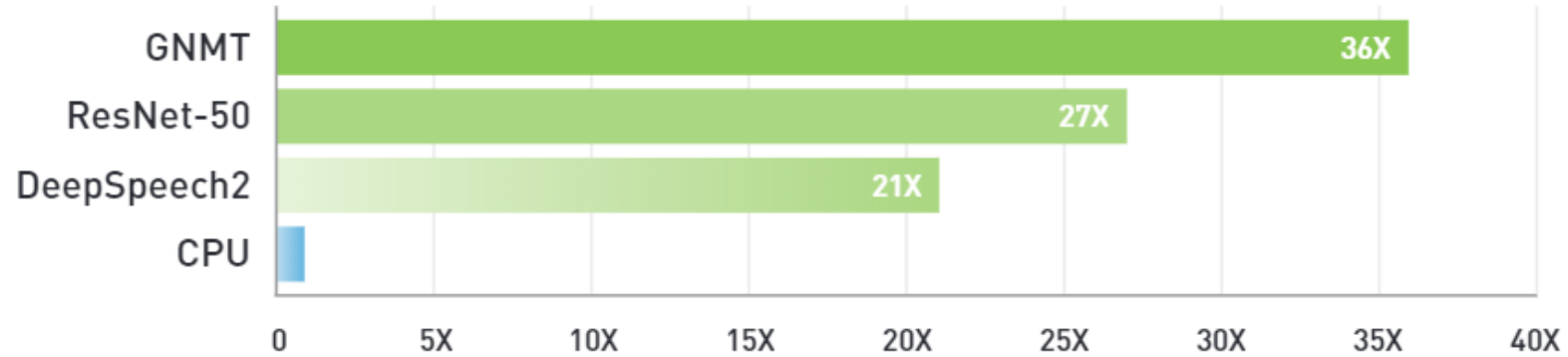
Jetson
Nano

...

DGX
Station

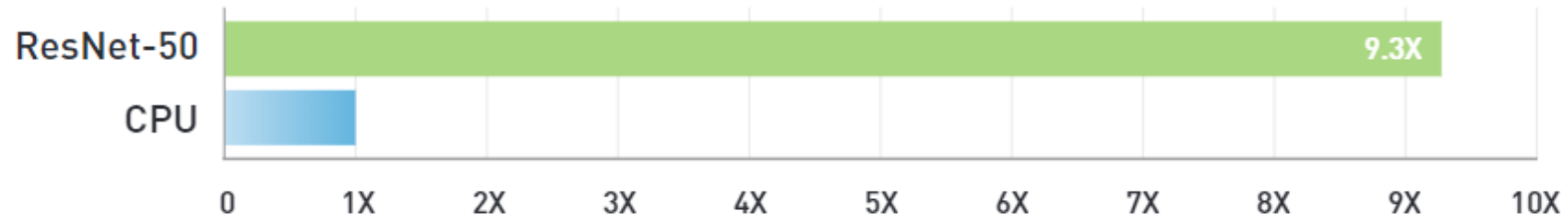


Inference Performance



Comparisons made of one NVIDIA Tesla T4 GPU and servers with a dual-socket Xeon Gold 6140 CPU.

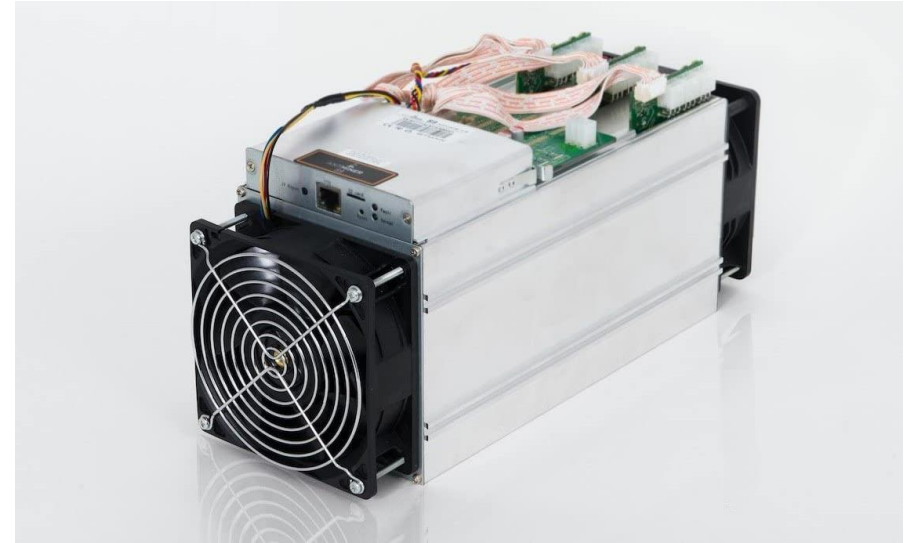
Training Performance



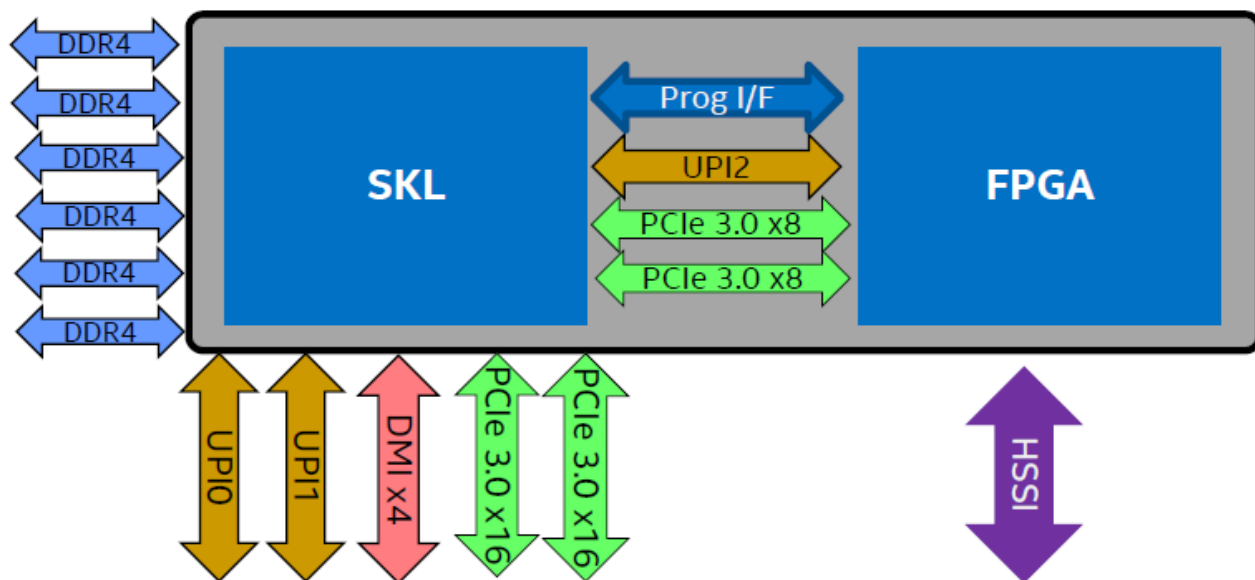
Comparison made between dual NVIDIA Tesla T4 GPUs and servers with a dual-socket Xeon Gold 6140 CPU.

ASIC & FPGA

- **Application Specific Integrated Circuit**
 - Altamente específico
 - E.g. mineros, AES
- **Field Programmable Gate Array**
 - Lógica combinatoria altamente adaptable
 - Velocidad más alta que una CPU en diferentes configuraciones
 - Experimentos de integración: Intel Skylake y otros



Skylake + FPGA on Purley



- Power for FPGA is drawn from socket & requires modified Purley platform specs
- Platform Modifications include Stackup, Clock, Power Delivery, Debug, Power up/down sequence, Misc IO pins (see BOM cost section)

Cores	Up to 28C with Intel® HT Technology	
FPGA	Altera® Arria 10 GX 1150	
Socket TDP	Shared socket TDP Up to 165W SKL & Up to 90W FPGA	
Socket	Socket P	
Scalability	Up to 2S – with SKL-SP or SKL + FPGA SKUs	
PCH	Lewisburg: DMI3 – 4 lanes; 14xUSB2 ports Up to: 10xUSB3; 14xSATA3, 20xPCIe*3 New: Innovation Engine, 4x10GbE ports, Intel® QuickAssist Technology	
	For CPU	For FPGA
Memory	6 channels DDR4 RDIMM, LRDIMM,	Low latency access to system memory via UPI & PCIe interconnect
	2666 1DPC, 2133, 2400 2DPC	
Intel® UPI	2 channels (10.4, 9.6 GT/s)	1 channel (9.6 GT/s)
PCIe*	PCIe* 3.0 (8.0, 5.0, 2.5 GT/s)	PCIe* 3.0 (8.0, 5.0, 2.5 GT/s)
	32 lanes per CPU Bifurcation support: x16, x8, x4	16 lanes per FPGA Bifurcation support: x8
High Speed Serial Interface (Different board design based on HSSI config)	N/A	2xPCIe 3.0 x8
		Direct Ethernet (4x10 GbE, 2x40 GbE, 10x10 GbE, 2x25 GbE)

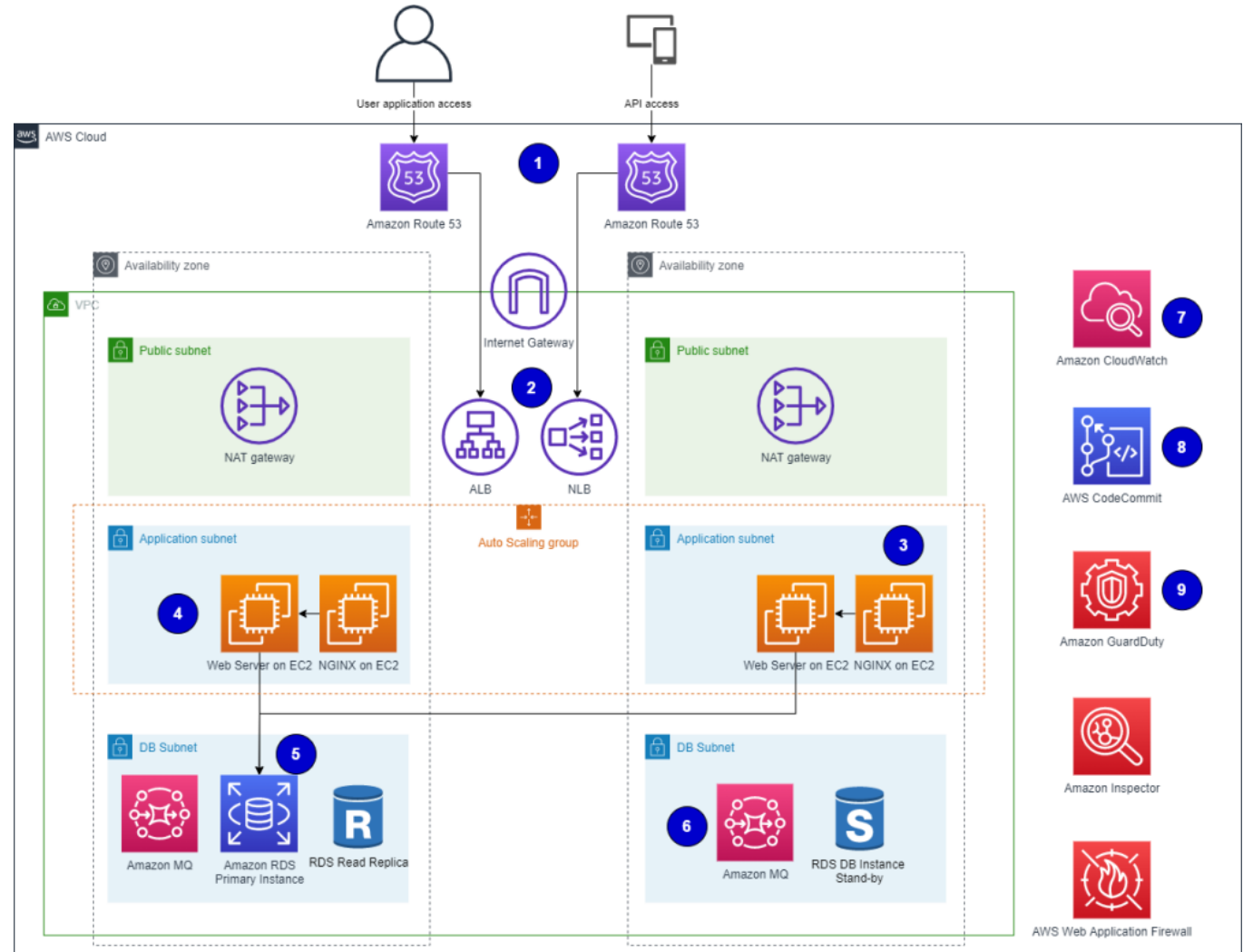
Código, algoritmos y estructuras de datos

- Evitar malas prácticas
 - Mal OOP, código innecesario, falta de Testing, abstracciones innecesarias
- Escoger estructuras correctas según el caso
- Stack excesivamente poblado
- Abstracciones innecesarias
 - ¡Ojo con cohesión, connascence y acoplamiento!



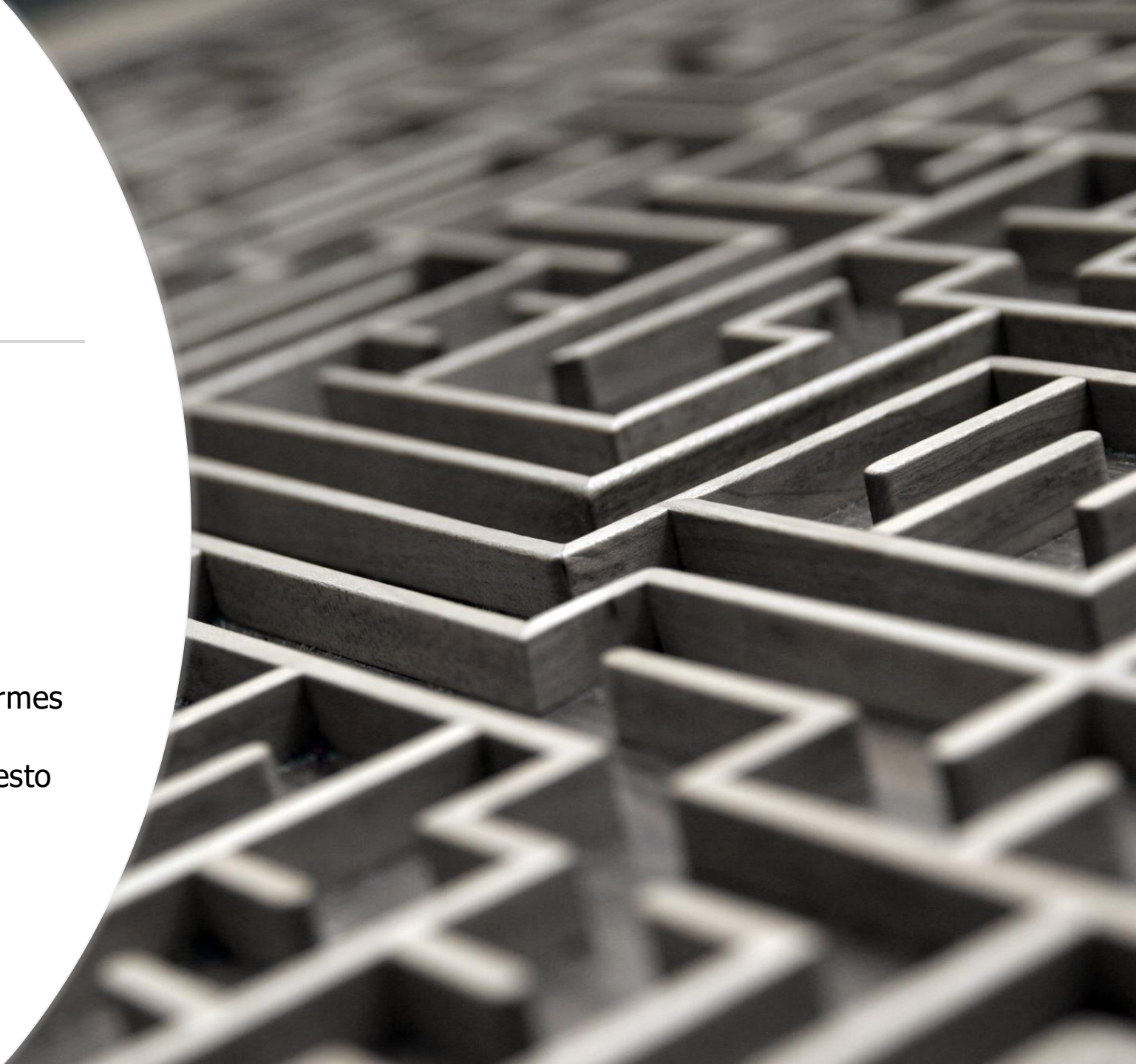
Red y performance

- Ancho de banda + latencia
 - Entre servidores y a usuarios
- Congestión
- Pérdida de paquetes
 - Incluso intracloud
- Escenarios
 - Cloud: **Replicación y redundancia** puede afectar
 - Consumo excesivo de red



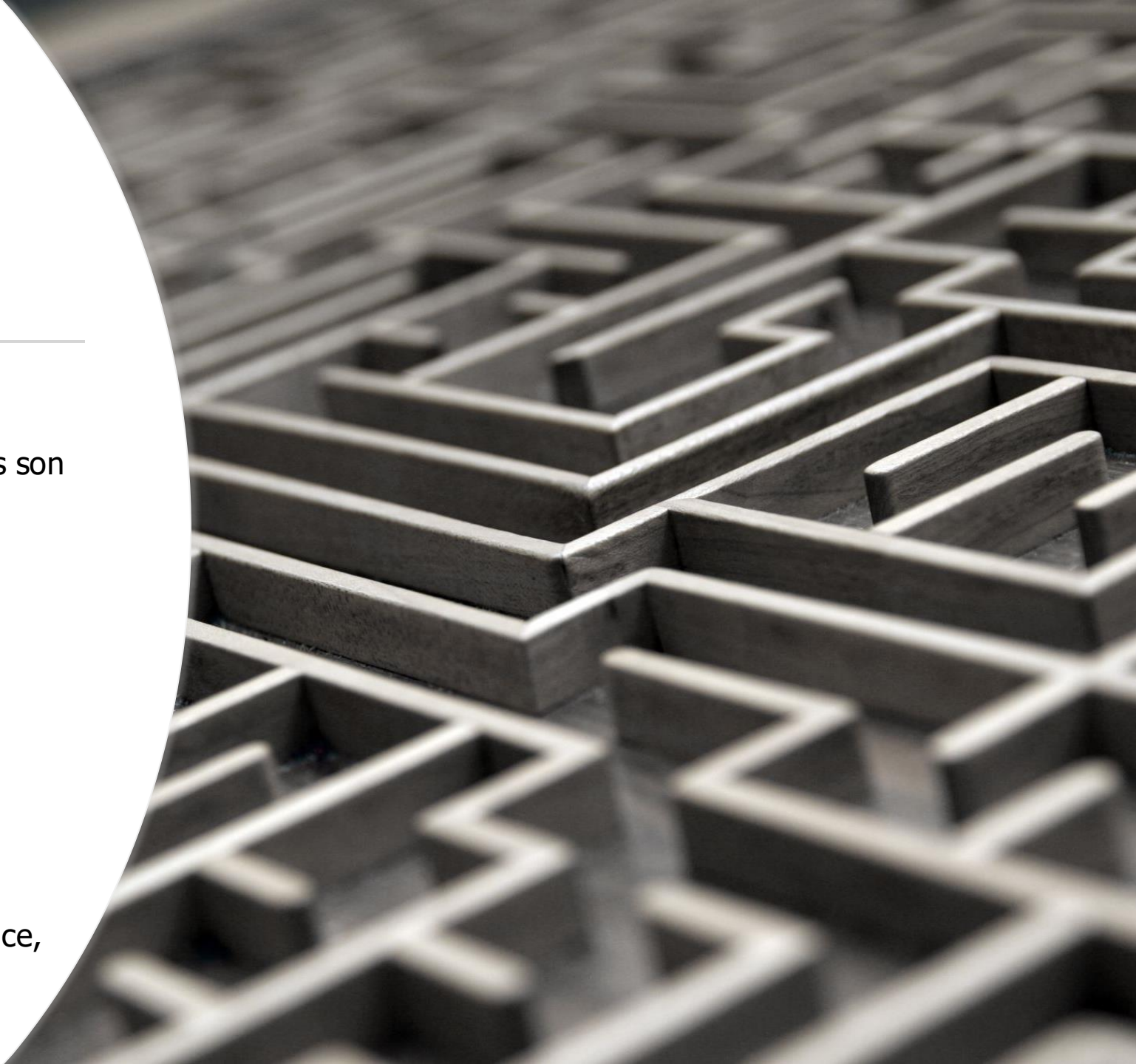
Problemas de la red (FOSA)

- La red no es confiable
 - Incluso en cloud puede perder paquetes
 - Instancias pueden caer repentinamente
 - Respuestas pueden perderse
- Latencia no es cero
 - Induce problemas de concurrencia
 - Encadenar requests puede ocasionar enormes efectos
 - SAGA y mediator pueden ocasionar esto mal configurados
- Ancho de banda no es infinito
 - Especialmente al salir del monolito



Problemas de la red (FOSA)

- La red no es segura
 - No podemos asumir que todos los actores son confiables
 - Más aún si salimos a la internet
 - Requerimos *firewalls* para apuntar a esto
- La topología no es fija
 - El movimiento de datos puede cambiar
 - El *routing* puede ser impredecible
- Costo de transporte > 0
 - En la nube es más explícito
- Red no homogénea
 - Los dispositivos varían en tipo, performance, vendor



Caso: Amazon Prime pasó de Microservicios a “Monolito”

- Sistema de monitoreo en tiempo real de reproducciones en *streaming* para detectar problemas de latencia, corrupciones, mal sincronización o esperas muy largas
- Problema:
 - Mucho tiempo de orquestar en la red
 - Se enviaban videos en la red para usarse en sistemas desacoplados, generaba mucha latencia y altos costos de recursos y dinero

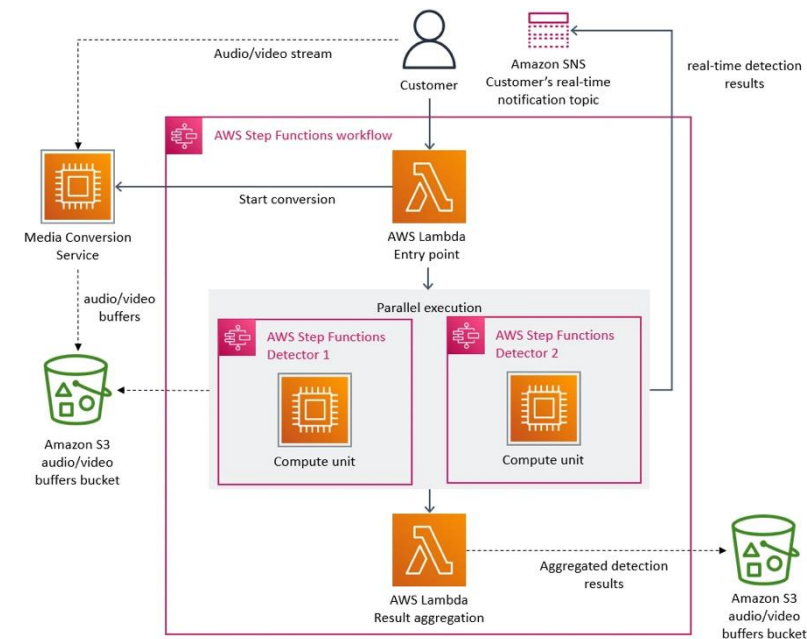
Return of the Monolith: Amazon Dumps Microservices for Video Monitoring

A blog post from the engineering team at Amazon Prime Video has been roiling the cloud native computing community with its explanation that, at least in the case of the video monitoring, a monolithic architecture has produced superior performance over a microservices and serverless-led approach.

May 4th, 2023 7:23am by Joab Jackson

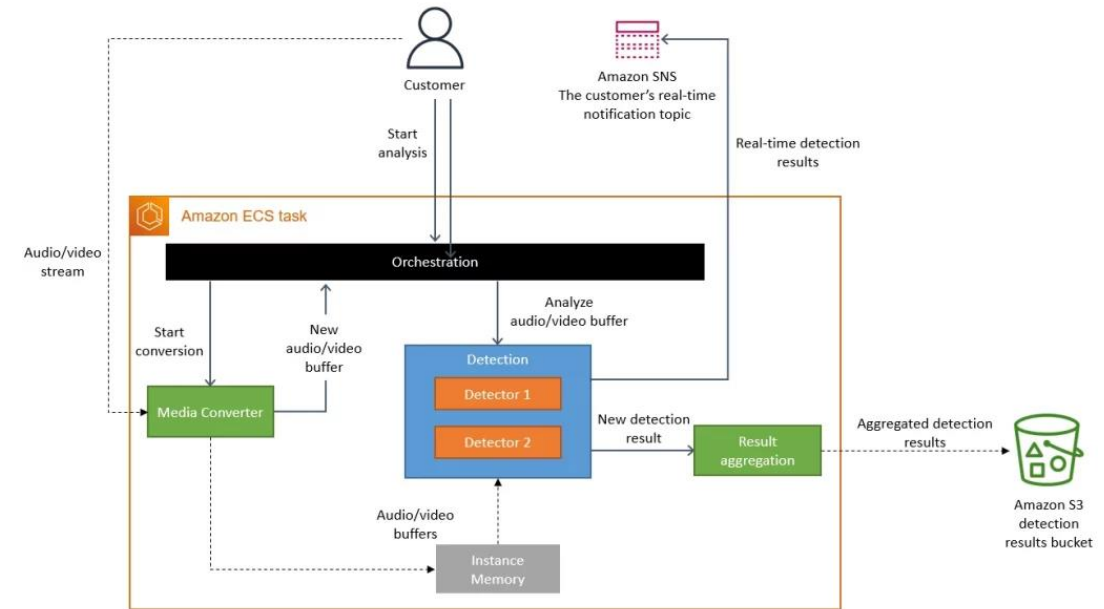


Image via Pexels.



Caso: Amazon Prime pasó de Microservicios a “Monolito”

- Solución Inicial:
 - Mover parte de su sistema a un monolito para no enviar todo por la red (solo se movió esa parte, el resto siguió distribuido)
 - Reducción de un 90% del costo
- Solución final:
 - Optimización de su flujo para no depender del envío en la red para archivos pesados
 - Aprovechar sus funciones serverless con el flujo de recepción de videos



Amazon Prime Video's Microservices Move Doesn't Lead to a Monolith after All

The streaming service provider made waves when its engineers reported they had refactored their QoS monitor for a monolithic architecture. Microservices experts evaluating the details discovered they actually did just the opposite.

Jun 13th, 2023 6:00am by Scott M. Fulton III



Feature image of Foucault's Pendulum from the National Museum of Science and Technology in Milan by Ben Ostrowsky, licensed under Public Domain

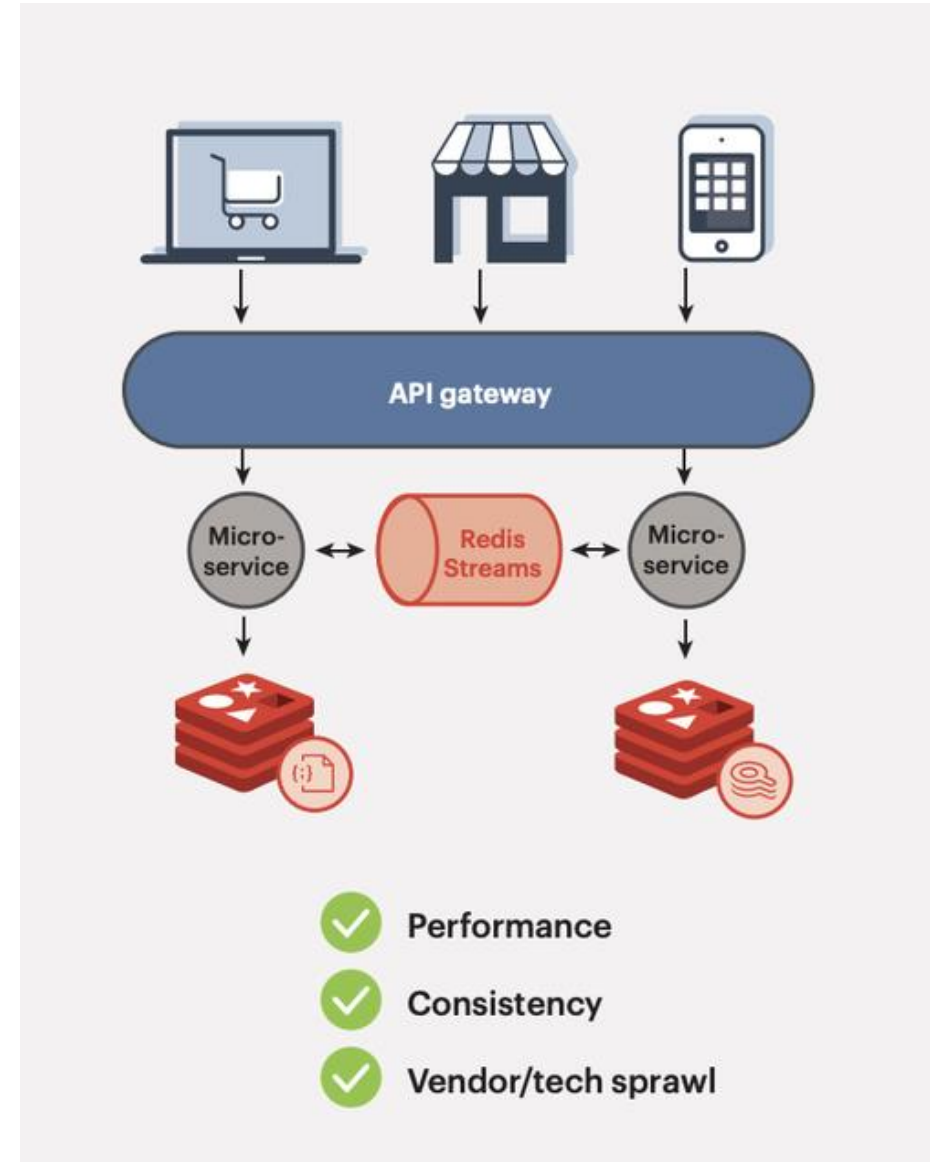
Caching

- Replicación de datos en diversas instancias para elevar performance
- Almacenes de objetos y/o valores
- Requerimientos de caching
 - Por proximidad
 - Por latencia
 - Por tamaño



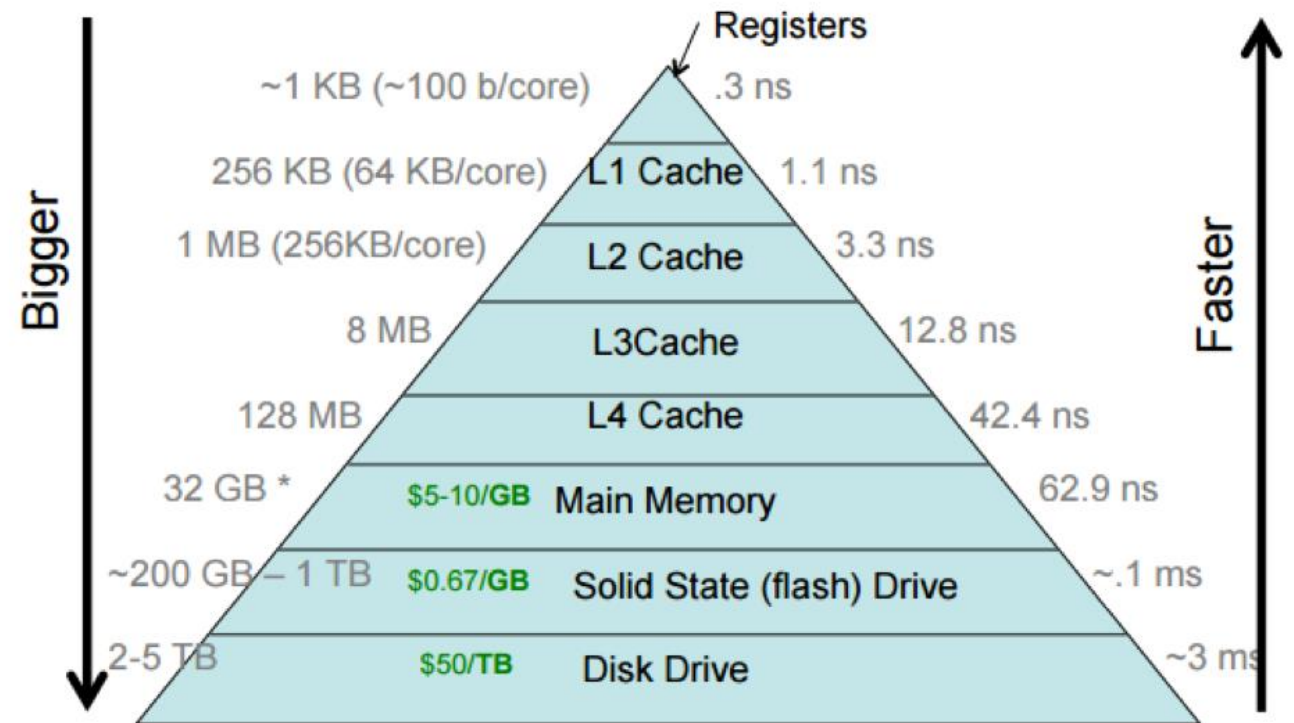
Desafíos del cache

- Sincronización y consistencia
 - Estrategias de invalidación
 - Tamaños (a veces usa RAM)
- Coherencia
 - Misma información en múltiples nodos
 - Varios servicios podrían no tener la misma cache (consistencia eventual)
- *Warm-up*
 - Cuando el sistema parte podría no tener muchos datos
- Complejidad
 - Implementar caché no es gratuito



Caching: CPU

- Arquitectura de capas (niveles)
- Cada nivel es más lento que el anterior
- Requerido por diferencias de velocidad con RAM

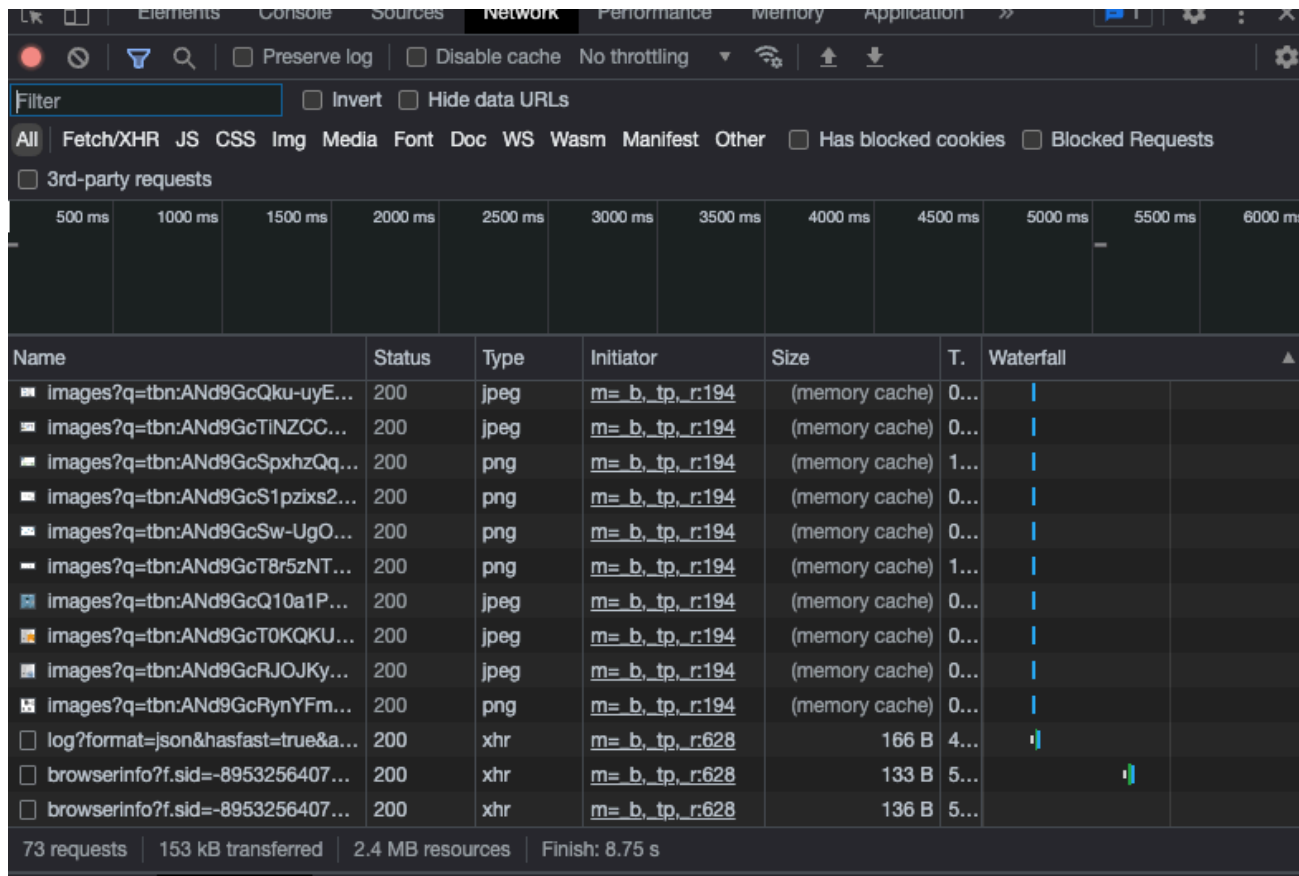


Caching: Web

En lugar de descargar contenido desde un servidor remoto, los navegadores lo descargan de un servidor geográficamente cercano o conectado por un enlace de red rápida (corto)

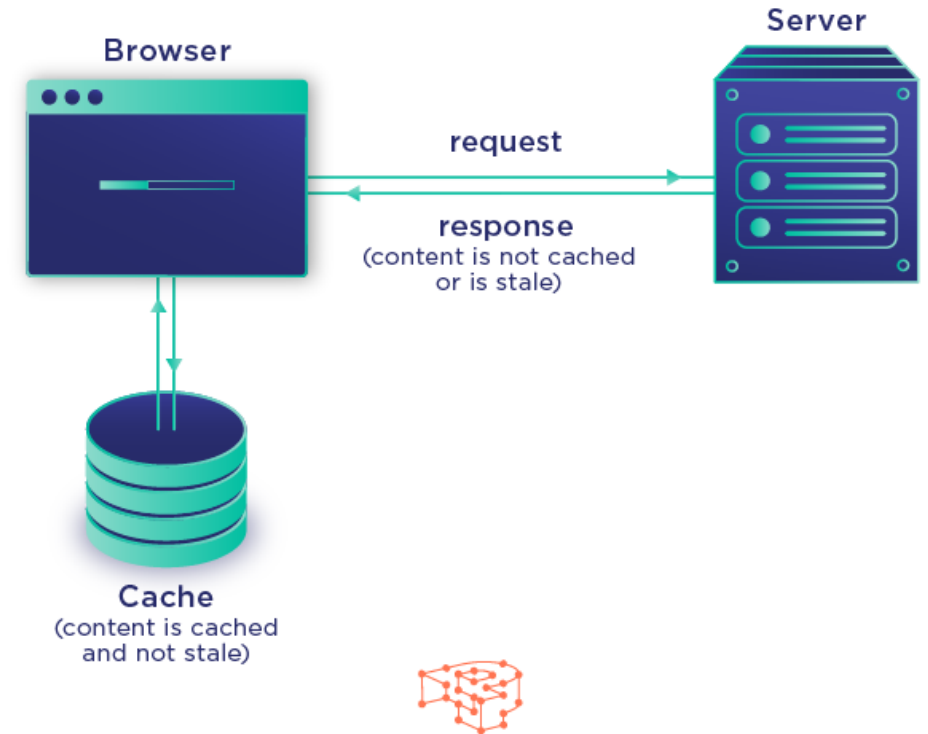
Almacenamiento en caché del navegador	Almacenamiento en caché de una red de distribución de contenido	Almacenamiento en caché de una red
Copias cercanas y rápidas, pero descargadas por un sólo usuario	Usado por empresas (<i>Webmasters</i>) que quieren que su contenido sea accesible rápidamente en todas partes <i>CDN, Reverse Proxy, Gateway Cache, Surrogate cache</i>	Copias cercanas por cada usuario que descargó el contenido (ahorra ancho de banda) <i>Proxy cache, Interception Proxy</i>

Caching: Web + Navegador



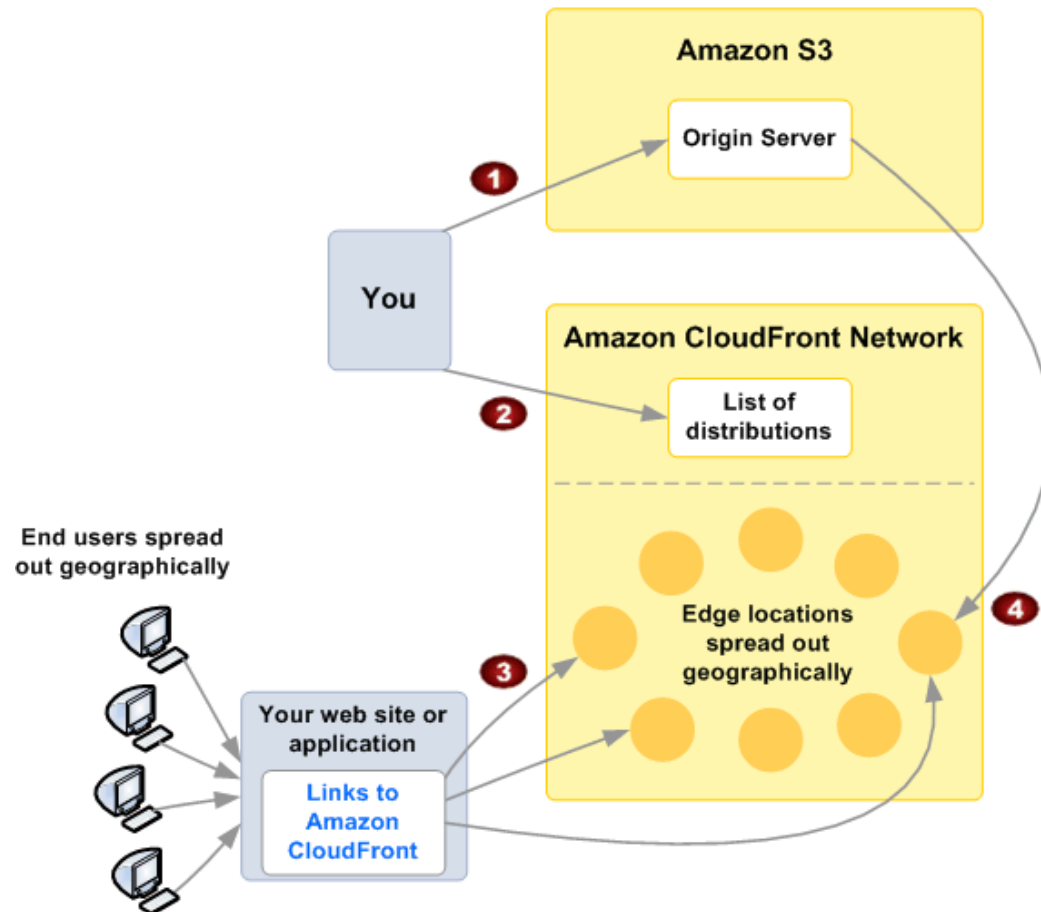
Name	Status	Type	Initiator	Size	T.	Waterfall
images?q=tbN:ANd9GcQku-uyE...	200	jpeg	m= b, tp, r:194	(memory cache)	0...	
images?q=tbN:ANd9GcTINZCC...	200	jpeg	m= b, tp, r:194	(memory cache)	0...	
images?q=tbN:ANd9GcSpxhzQq...	200	png	m= b, tp, r:194	(memory cache)	1...	
images?q=tbN:ANd9GcS1pzixs2...	200	png	m= b, tp, r:194	(memory cache)	0...	
images?q=tbN:ANd9GcSw-UgO...	200	png	m= b, tp, r:194	(memory cache)	0...	
images?q=tbN:ANd9GcT8r5zNT...	200	png	m= b, tp, r:194	(memory cache)	1...	
images?q=tbN:ANd9GcQ10a1P...	200	jpeg	m= b, tp, r:194	(memory cache)	0...	
images?q=tbN:ANd9GcT0KQKU...	200	jpeg	m= b, tp, r:194	(memory cache)	0...	
images?q=tbN:ANd9GcRJOJKy...	200	jpeg	m= b, tp, r:194	(memory cache)	0...	
images?q=tbN:ANd9GcRynYFm...	200	png	m= b, tp, r:194	(memory cache)	0...	
log?format=json&hasfast=true&a...	200	xhr	m= b, tp, r:628	166 B	4...	
browserinfo?f.sid=-8953256407...	200	xhr	m= b, tp, r:628	133 B	5...	
browserinfo?f.sid=-8953256407...	200	xhr	m= b, tp, r:628	136 B	5...	

73 requests | 153 kB transferred | 2.4 MB resources | Finish: 8.75 s

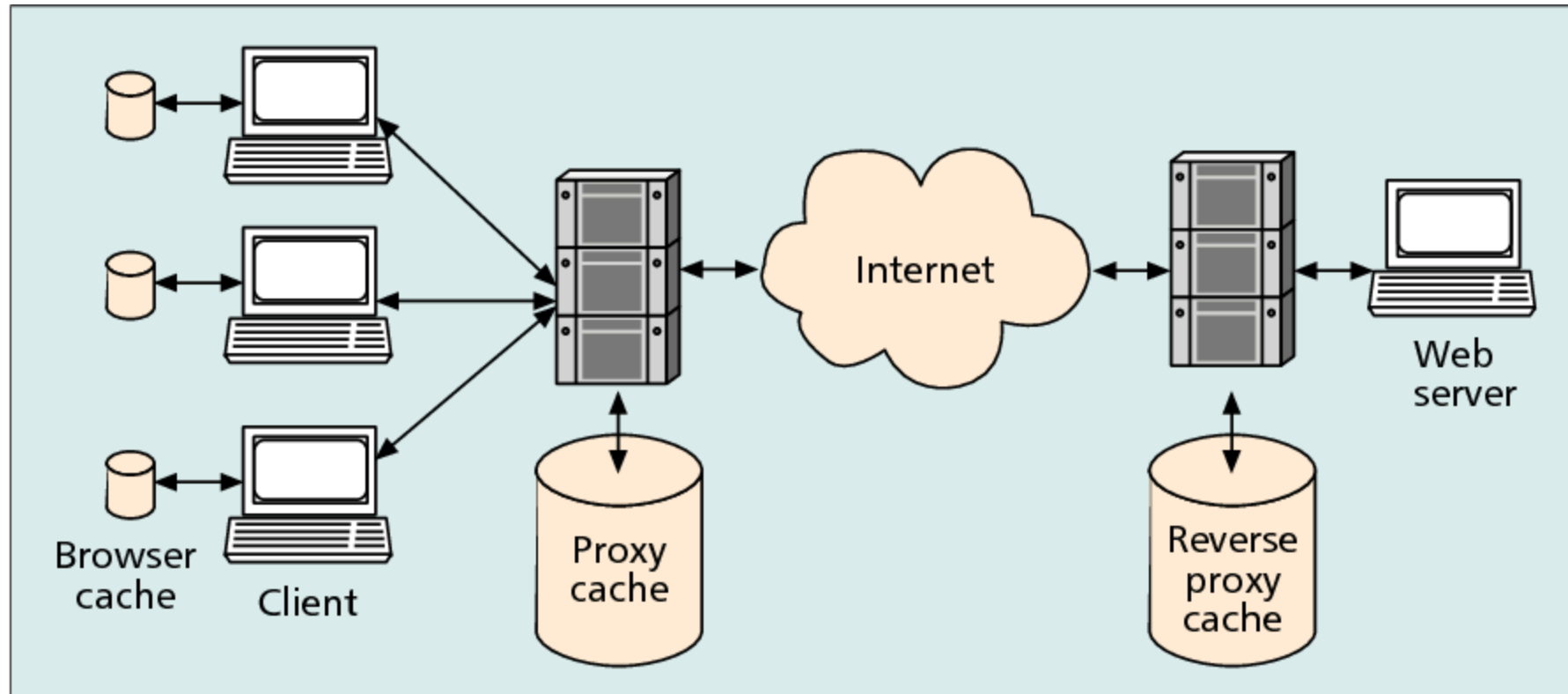


Caching: Web + CDN

- *Content Delivery Network*
- Para archivos estáticos
 - Fotos
 - Videos
 - Audio
 - Streaming
- Aprovecha **cercanía y latencia para mejorar el servicio**
- **Ojo:** Debe **manejarse la consistencia**
 - Una mala *cache policy* o no usar *etags* podría servir contenido viejo (*stale*)
- En su proyecto: Distribución Cloudfront



Caché en proxy inverso





Performance III

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot

Reglas de caché en la web

- HTTP 1.0 / 1.1
- Si el response Header dice No-Cache, no se cachea
 - Si el request es autenticado o seguro (HTTPS), no se cachea
 - Una representación cacheada es fresca (se sirve del cache) si:
 - Tiene **fecha de expiración o header Age-Control < actual**
 - El **cache ha visto la representación hace poco** y fue **modificada hace tiempo** (*Last-Modified*)
- Si una representación es añeja (*stale*), se pedirá al servidor de origen validarla
- Bajo ciertas circunstancias (*offline*) se servirá la respuesta añeja sin chequear con el servidor de origen

Implementación de cache

- HTML Headers: no es una buena idea (sólo afecta a algunos browsers)
- HTTP Headers
- **Expires** (*Response*):
 - Afecta a todos los caches (pero Cache-Control tiene precedencia)
 - Fecha absoluta, última fecha de acceso, o fecha de última modificación
 - Sincronización de relojes
 - Problema: ¡Si no se actualiza el Expires, hay un aumento de visitas!

```
HTTP/1.1 200 OK
Date: Fri, 30 Oct 1998 13:19:41 GMT
Server: Apache/1.3.3 (Unix)
Cache-Control: max-age=3600, must-revalidate
Expires: Fri, 30 Oct 1998 14:19:41 GMT
Last-Modified: Mon, 29 Jun 1998 02:28:12 GMT
ETag: "3e86-410-3596fbbc"
Content-Length: 1040
Content-Type: text/html
```

Buenas prácticas en la web para el caché

- Usar URLs de manera consistente
- Usar librerías de imágenes comunes
- Marcar imágenes y páginas que no cambian a menudo con *Cache-Control: max-age* y un valor grande
- Ajustar el valor de *max-age* o *Expires* apropiado para los otros recursos
- No actualice archivos de manera innecesaria (falso *Last-Modified*)
- Usar cookies cuando sea necesario (son difíciles de cachear, sólo usar en páginas dinámicas)

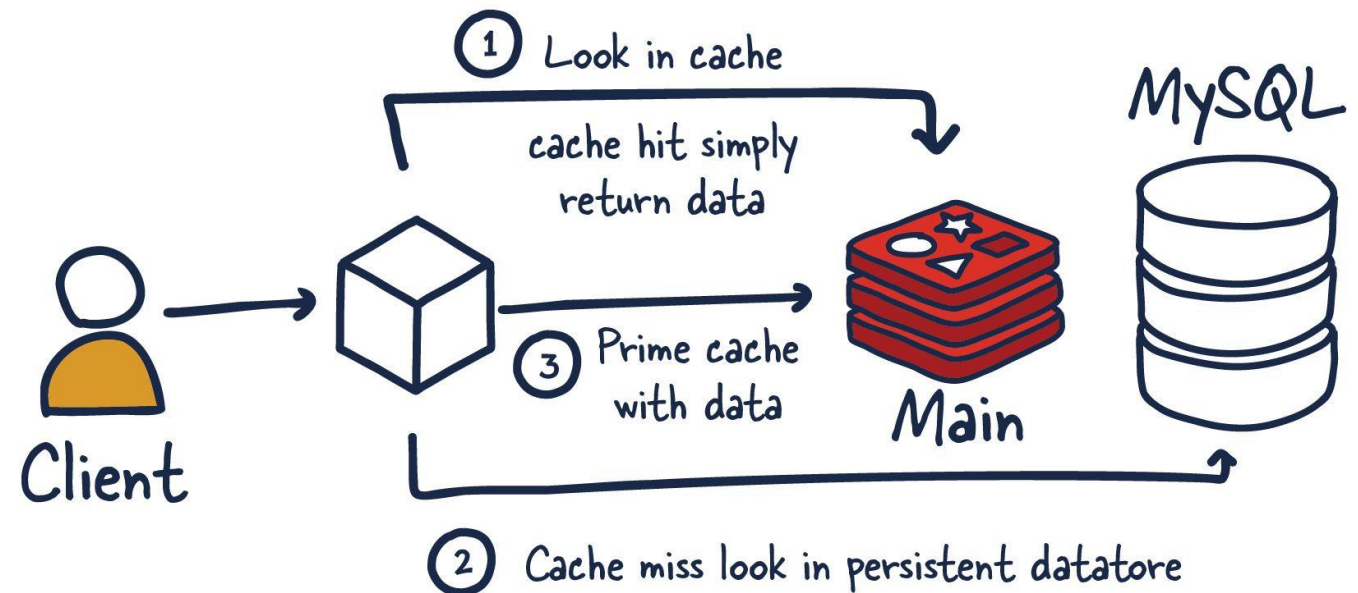
Buenas prácticas para el caché en general

- Entender **cuáles datos necesitan caching**
 - Archivos estáticos
 - Consultas repetidas
- Políticas de desalojo (*eviction*) y expiración de caché
 - Valores apropiados
 - Elección LRU, LFU, FIFO
- Caché jerárquico
 - Múltiples capas aprovechan fortalezas únicas

Buenas prácticas para el caché en general

- **Monitoree** su caché
- Escoja el *stack* de caché correcto
 - Calibre tecnologías
 - Ojo con software propietario/privativo
- **Prefetch** del caché
 - Mejora capacidad de inicio y respuestas predictivas

How is redis traditionally used



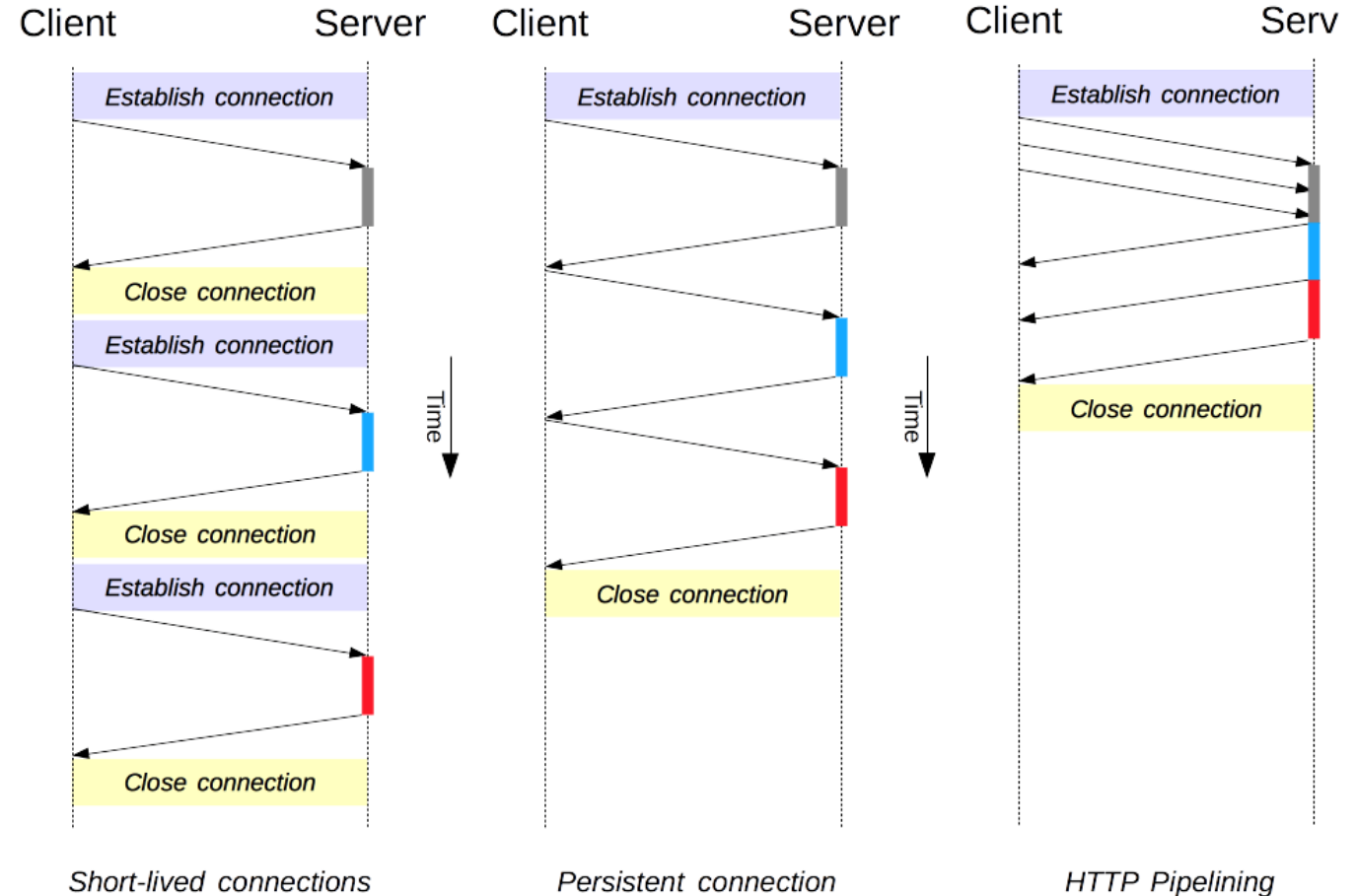


Stack: HTTP X

- HTTP/2.0 Segunda versión principal del protocolo de red HTTP
 - Originario de SPDY, Google
 - RFC9113
- Mejora características sustanciales de HTTP/1.1
 - Protocolo binario
 - Multiplexado
 - Server push
 - Compresión de headers

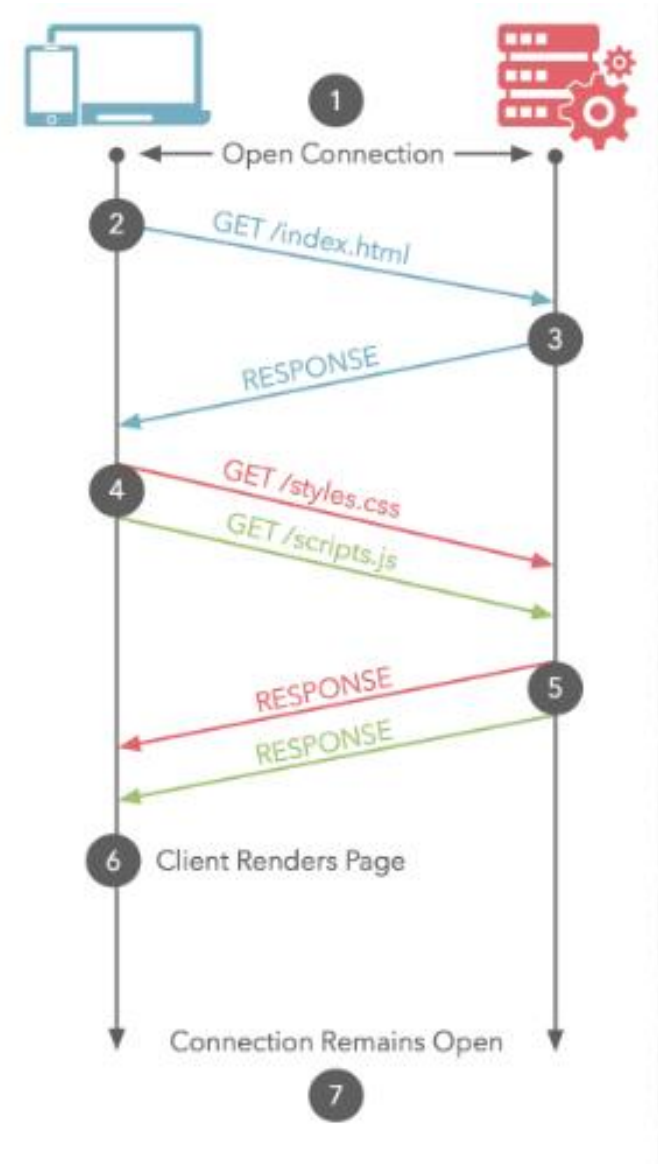
Caso HTTP/1.0

- HTTP/1.0: ¡Un *request* por conexión TCP!
- HTTP/1.0 con *Connection: keep-alive*
 - Se pueden hacer **varios requests uno después de otro**
 - Minimiza el costo de abrir conexiones
 - Es bloqueante: 1 outstanding request a la vez
- HTTP/1.1 con *Pipelining*
 - Request múltiples, ordenados
 - La **respuesta**, sin embargo, **sigue siendo bloqueante** (*Head Of Line, HOL*) y ordenada (*FIFO*)
 - Actualmente desactivado por defecto
- Browsers mantienen 4-8 conexiones



Caso HTTP/2.0

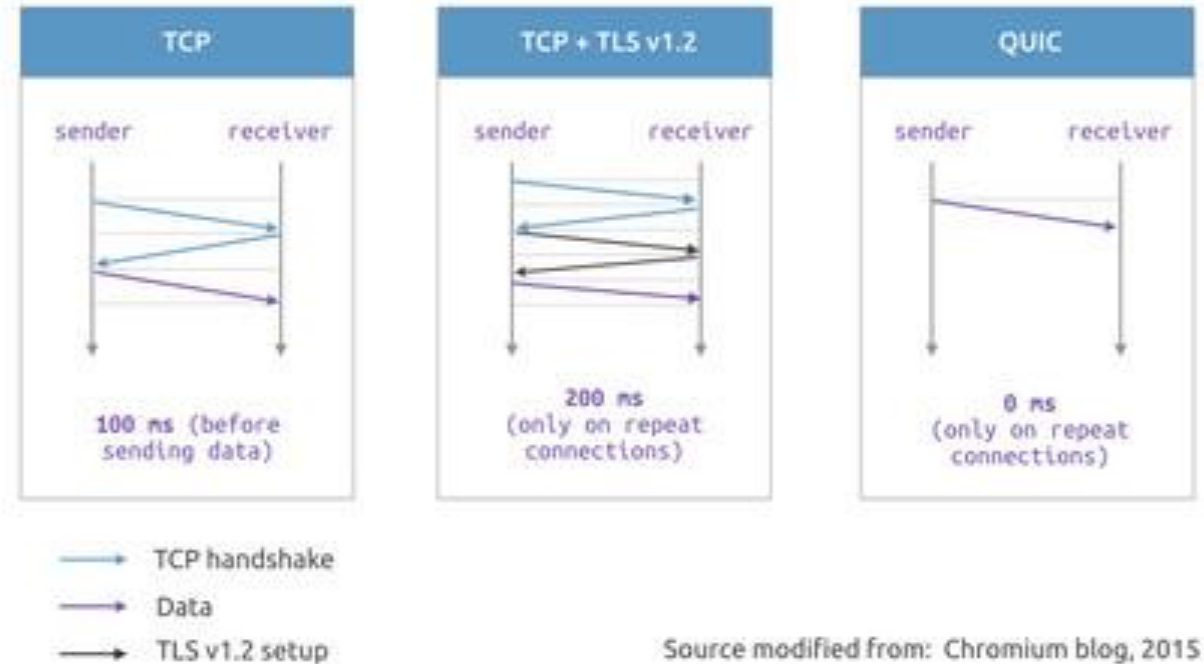
- Una **conexión multiplexada**
 - Varios requests
 - Priorizados
 - *Out Of Order*
 - Requests entrelazadas
- Compresión de headers
- Server Push
 - Llenado preventivo de datos



Stack: HTTP 3.0

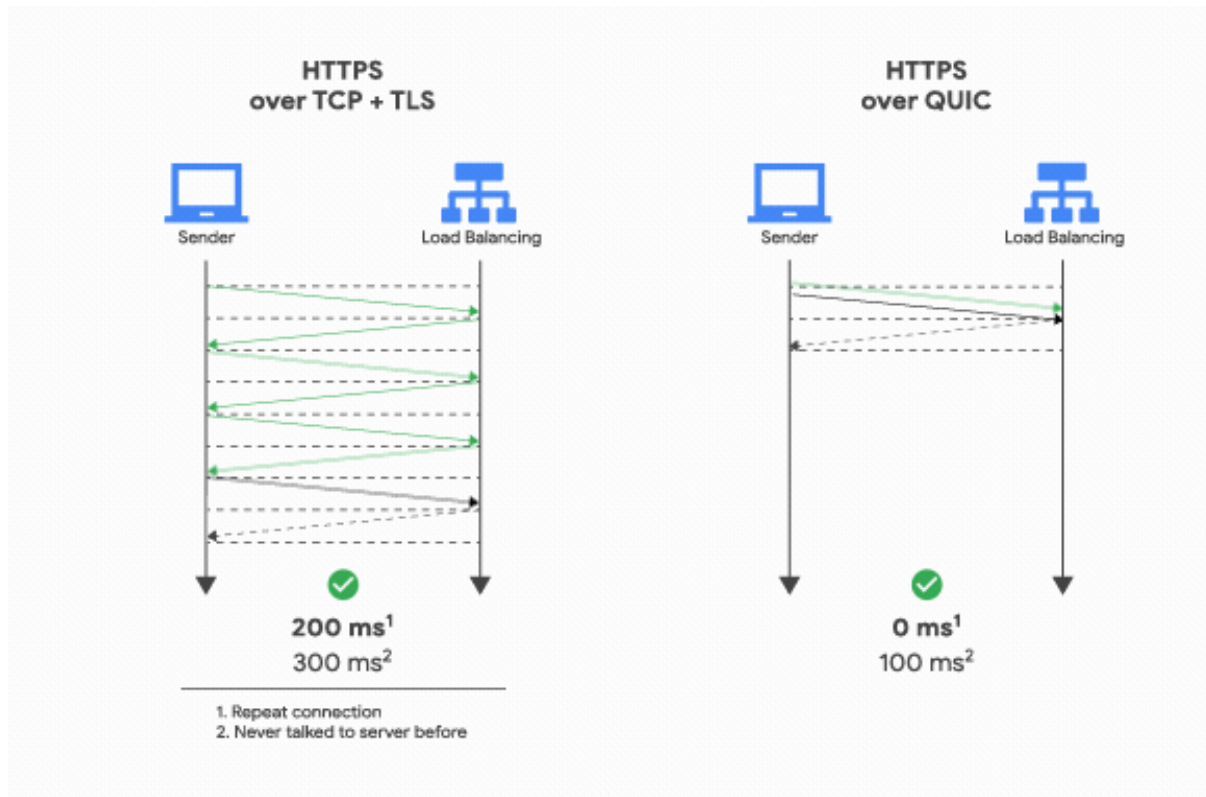
- Tercera versión del protocolo de red HTTP
 - Basado en *Quick UDP Internet Connections* (QUIC)
 - RFC9114
- Más mejoras
 - Mutiplexado avanzado
 - Uso de **UDP**
 - Cifrado obligatorio prenegociado
 - Recuperación de errores
- Ganancias
 - <https://cloudplatform.googleblog.com/2018/06/Introducing-QUIC-support-for-HTTPS-load-balancing.html>

Comparison between TCP, TCP + TLS, and QUIC

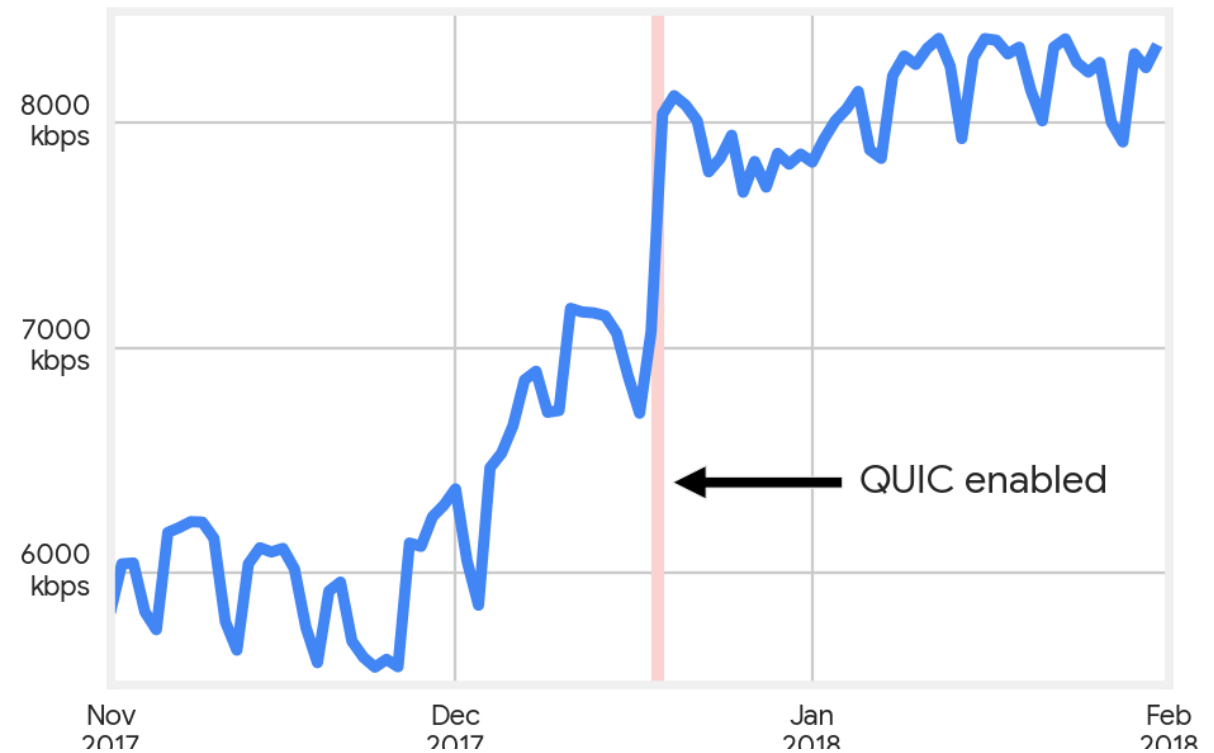


Source modified from: Chromium blog, 2015

Ganancias HTTP 3.0



Cloud CDN throughput (50th percentile)

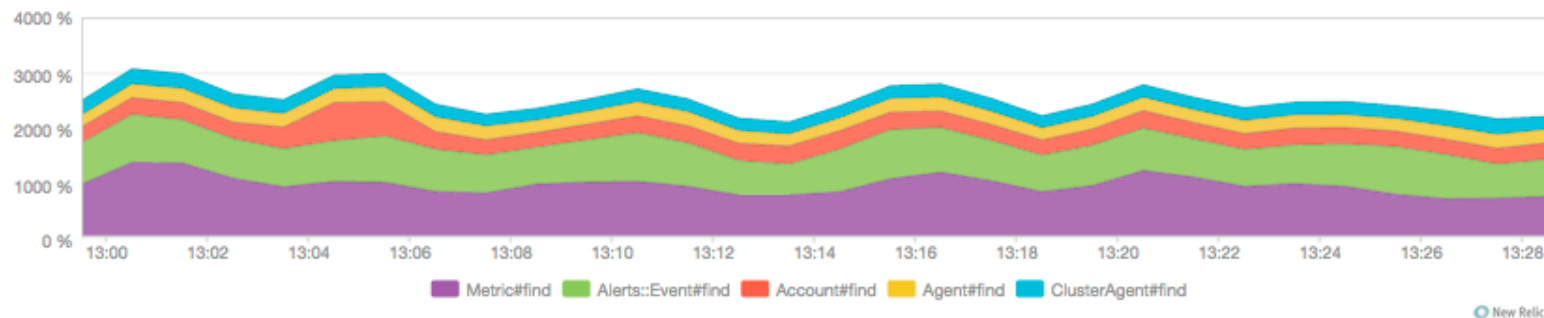


Monitoreo de datos: Métricas

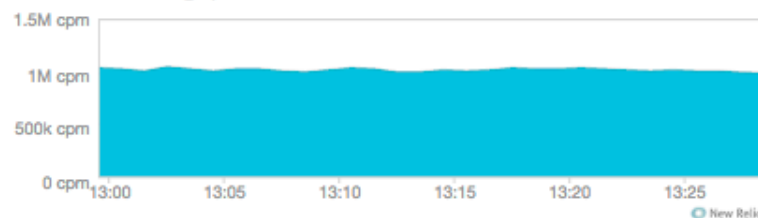
Sort by Most time consuming

Metric#find	17.4%
Alerts::Event#find	13.7%
Account#find	5.98%
Agent#find	4.46%
ClusterAgent#find	4.43%
SQL - SELECT	4.21%
SQL - OTHER	4.13%
Subscription#find	3.29%
SQL - UPDATE	3.09%
SQL - INSERT	2.94%
Deprecated::TransactionTrace#find	2.92%
AgentSummary#find	2.51%
TargetDowntimeEvent#find	2.35%
AccountView#find	2.13%
RealAgent#find	1.94%
ApplicationSettings#find	1.89%
ShardSpecification#find	1.69%
AccountProductLevel#find	1.53%

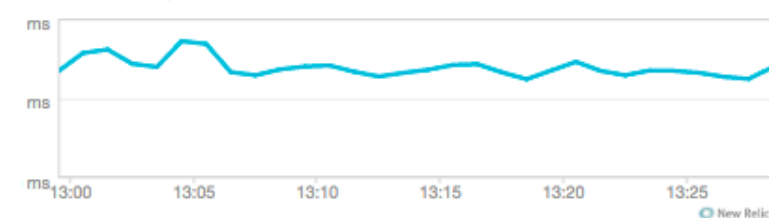
Top 5 database operations by wall clock time



Database throughput



Database response time

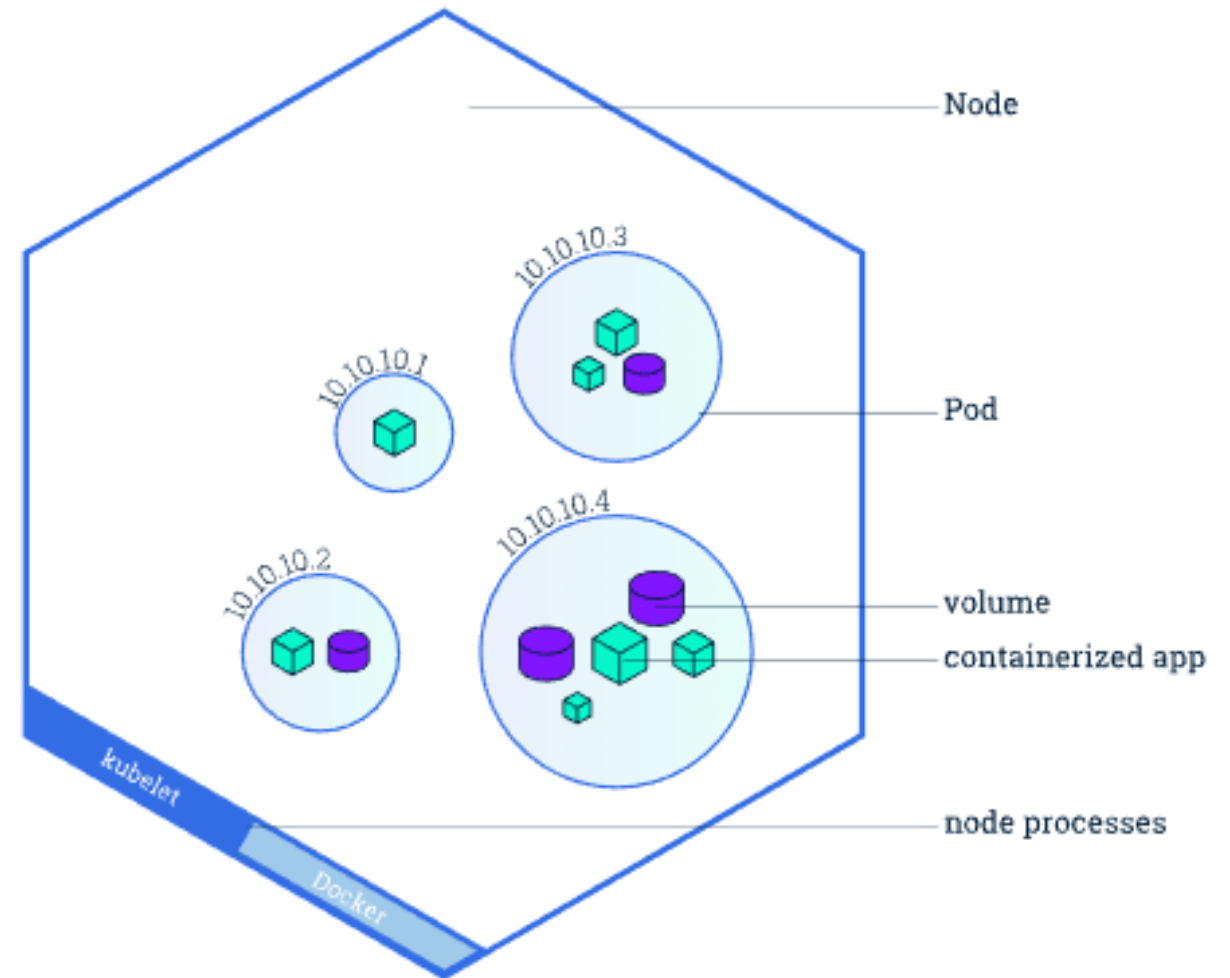


Slow SQL traces

	Response time	Sample count
SELECT max(id) id, path, exception_class, real_agent_id, sum(count) count, min(timestamp) min_ts...	78,400 ms	2
SELECT 'metrics'.* FROM 'metrics' INNER JOIN 'metrics_real_agents' ON 'metrics_real_agents'.met...	76,800 ms	5,286
SELECT 'agents'.* FROM 'agents' WHERE 'agents'.type = 'ClusterAgent' ORDER BY agents.created_a...	54,400 ms	10
SELECT accounts.id, accounts.name FROM 'accounts' INNER JOIN 'account_product_levels' ON 'account...	47,000 ms	5

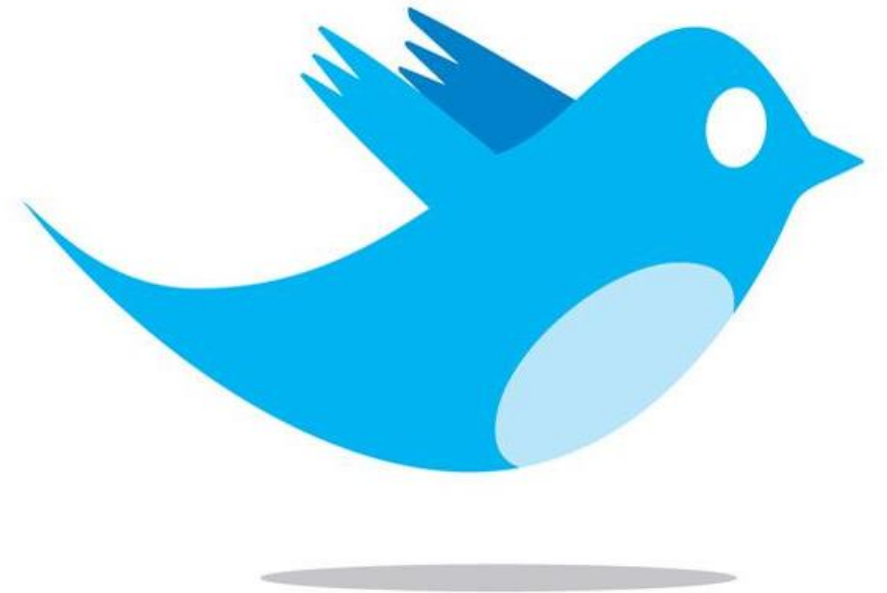
Stack: Containerización

- El uso de *containers* puede llevar a **mejor utilización de recursos**
- Diversas aplicaciones pueden compartir una *VM* o máquina
 - Puede **usarse toda la capacidad** de una máquina
 - La aislación permite *commoditizar* el hardware
- Mejora la escalabilidad

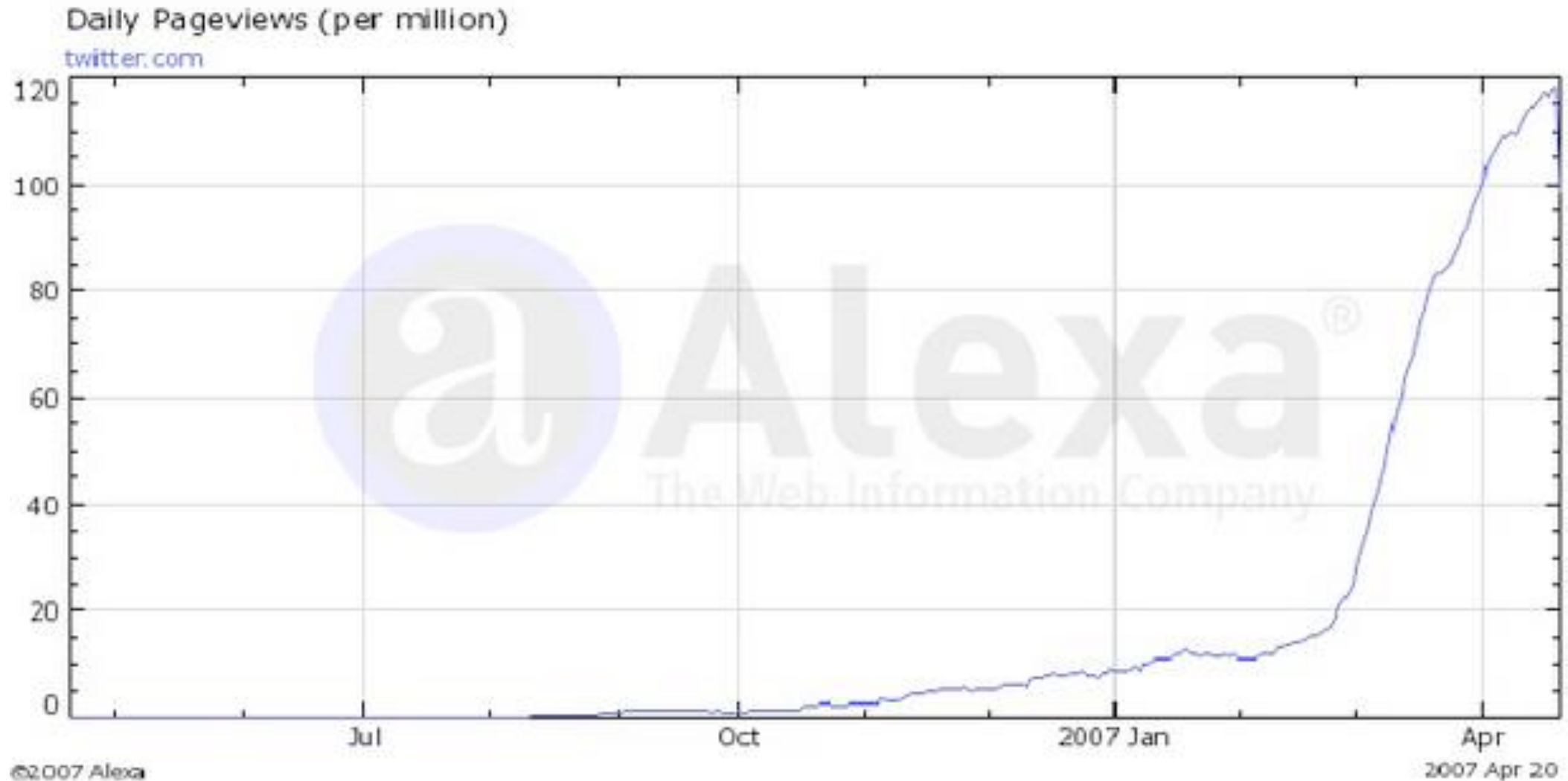


Caso Twitter (ahora X)

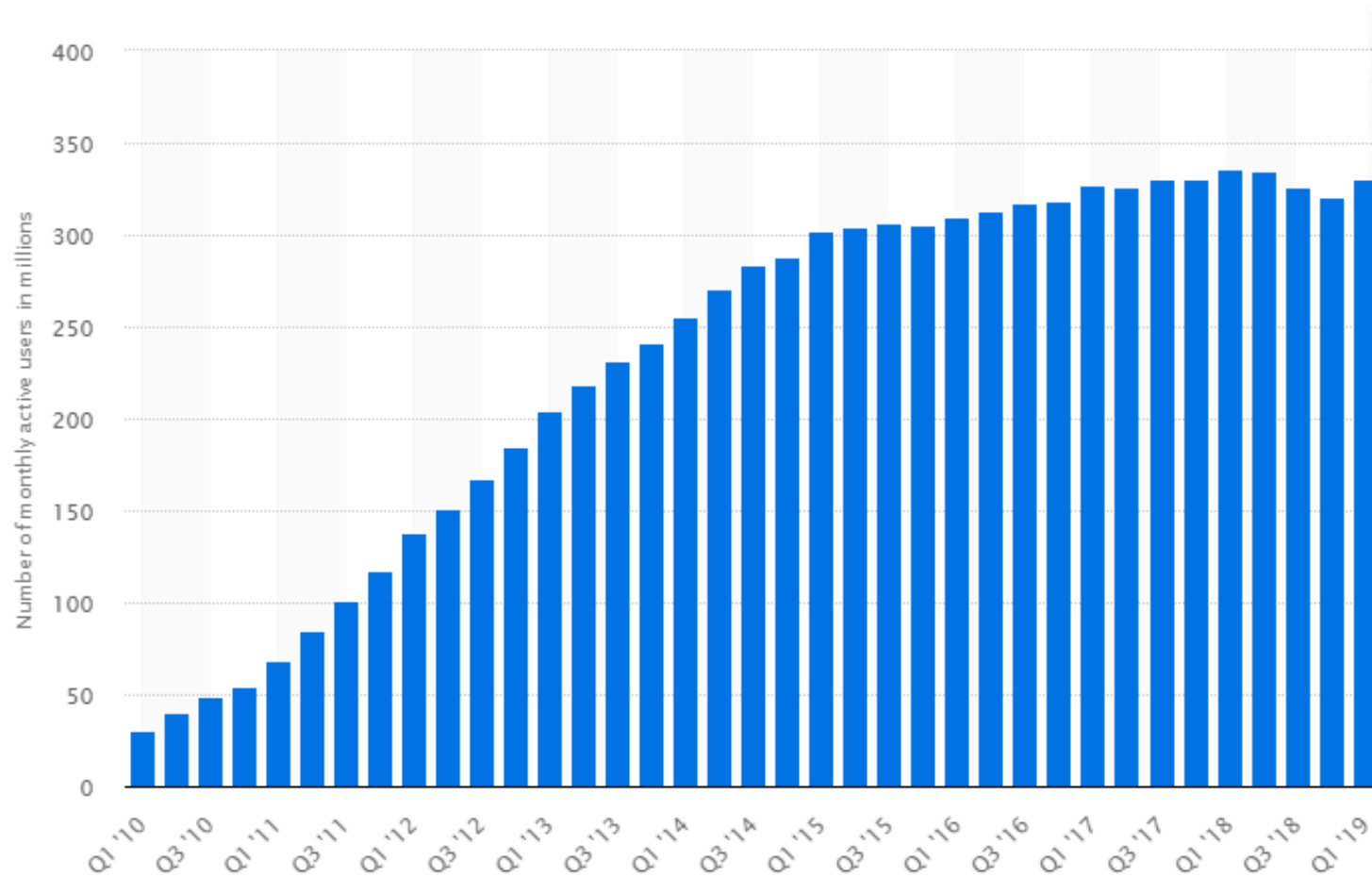
- 2007
 - 600 requests/segundo
 - 180 instancias Rails
 - 1 DB MySql + slave
 - 30 procesos varios
 - 8 máquinas Sun X4100s



Caso Twitter



Caso Twitter: MAUs



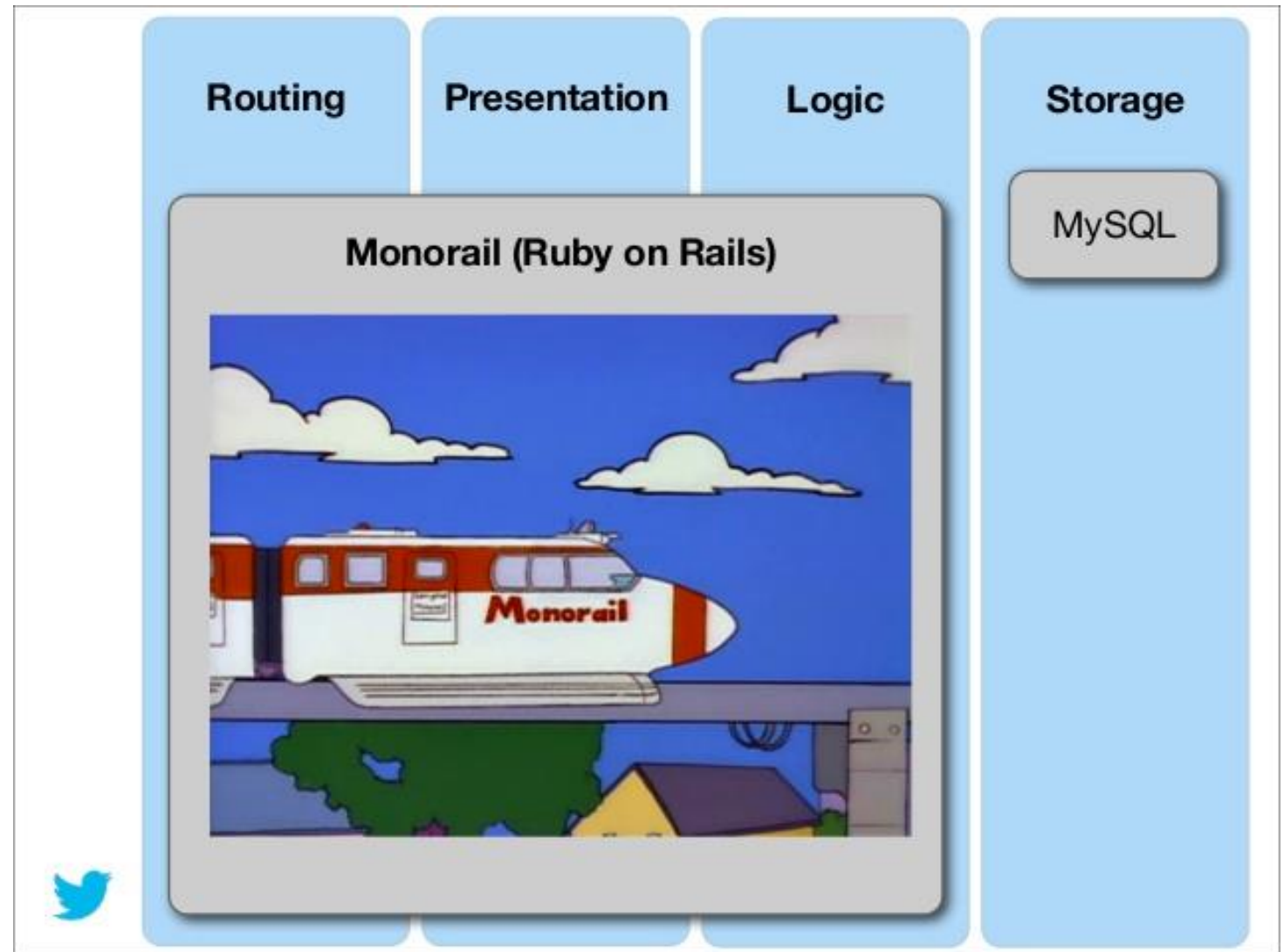
Twitter: 2015

- 2015
 - 500.000.000 Tweets/día (~600.000 Tweet/s [10x 2006])
 - 2013 record: 143,199 TPS
(<https://blog.twitter.com/2013/new-tweets-per-second-record-and-how>)
 - 100+ Proyectos de código abierto (open source)
 - 230M+ Usuarios activos
 - 75+ Lenguajes soportados



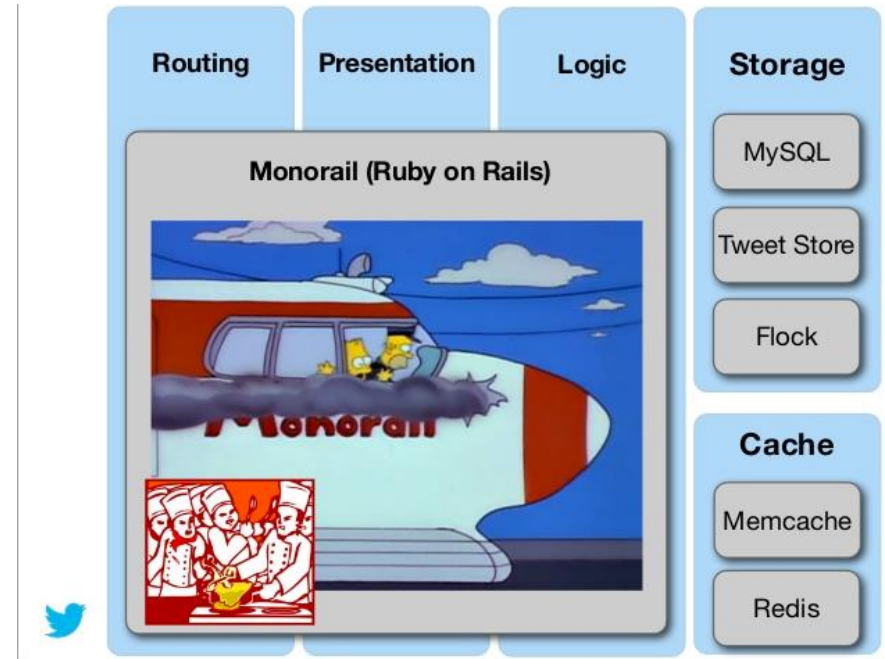
Estado inicial

- Inicialmente: Gran aplicación con Rails
 - Lanzando más máquinas a problema, en vez de dar una solución de ingeniería
 - Optimización de código vs legibilidad y Flexibilidad
 - Recursos de las máquinas siendo el cuello de botella



Estado inicial

- Inicialmente: Gran aplicación con Rails
 - Lanzando más máquinas a problema, en vez de dar una solución de ingeniería
 - Optimización de código vs legibilidad y Flexibilidad
 - Recursos de las máquinas siendo el cuello de botella



Twitter: Transición

- Cambio de Ruby VM a JVM (posteriormente a Scala)
 - Search (Java via Lucene)
 - FlockDB (Social Graph - Scala)
- Buena experiencia - Migrar todo a servicios aislados
 - Tweet service
 - User service
 - Timeline service
 - DM service
 - Social Graph Service

Decomposing the Monolith

Created services based on our core nouns:

Tweet service

User service

Timeline service

DM service

Social Graph service

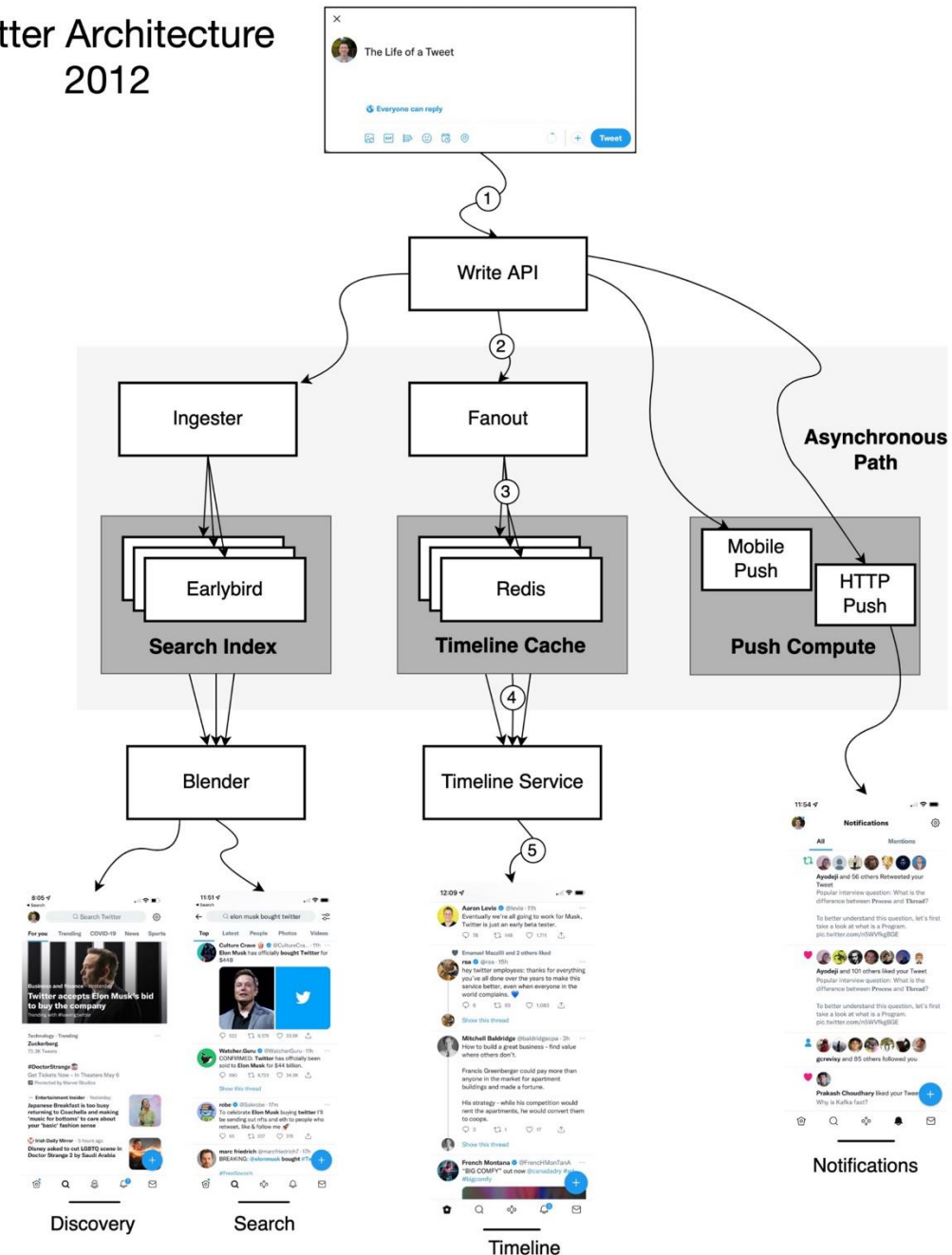
....



Twitter: Transición

- Cambio de Ruby VM a JVM (posteriormente a Scala)
 - Search (Java via Lucene)
 - FlockDB (Social Graph - Scala)
- Buena experiencia - Migrar todo a servicios aislados
 - Tweet service
 - User service
 - Timeline service
 - DM service
 - Social Graph Service

Twitter Architecture 2012



Elevar complejidad trae consecuencias

- Puede que recién llegados no entiendan
- 0 –
- Se arrastre complejidad



Elon Musk ✓
@elonmusk

Replying to @elonmusk and @sampullara

Part of today will be turning off the “microservices” bloatware. Less than 20% are actually needed for Twitter to work!

7:27 AM · Nov 14, 2022 · Twitter for iPhone

...



Elon Musk ✓
@elonmusk

...

Replying to @sampullara

I was told ~1200 RPCs independently by several engineers at Twitter, which matches # of microservices. The ex-employee is wrong.

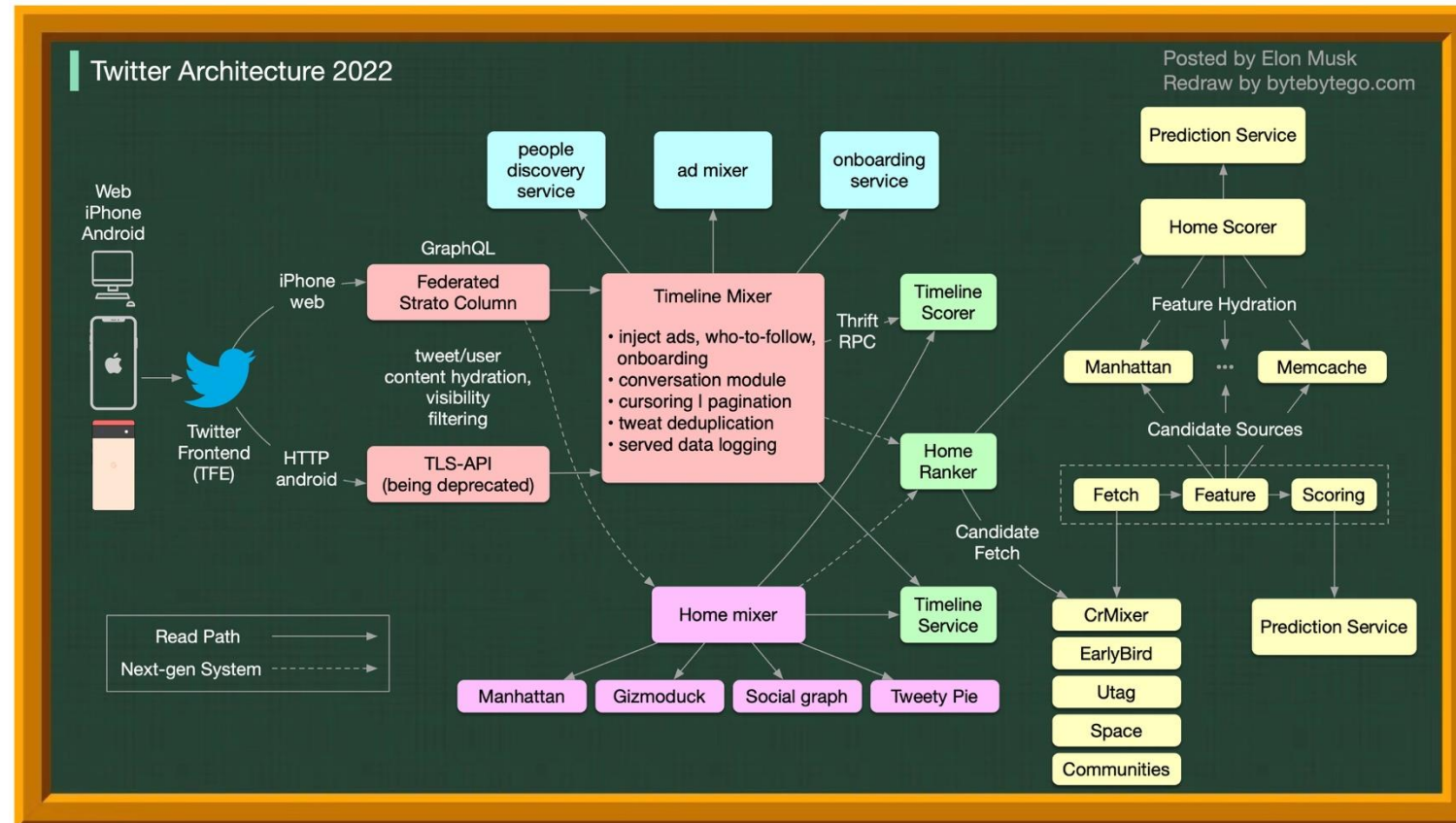
Same app in US takes ~2 secs to refresh (too long), but ~20 secs in India, due to bad batching/verbose comms. Actually useful data transferred is low.

8:52 PM · Nov 14, 2022 · Twitter for iPhone

449 Retweets 279 Quote Tweets 6,965 Likes

Elevar complejidad trae consecuencias

- Puede que recién llegados no entiendan
- 0 –
- Se arrastre complejidad



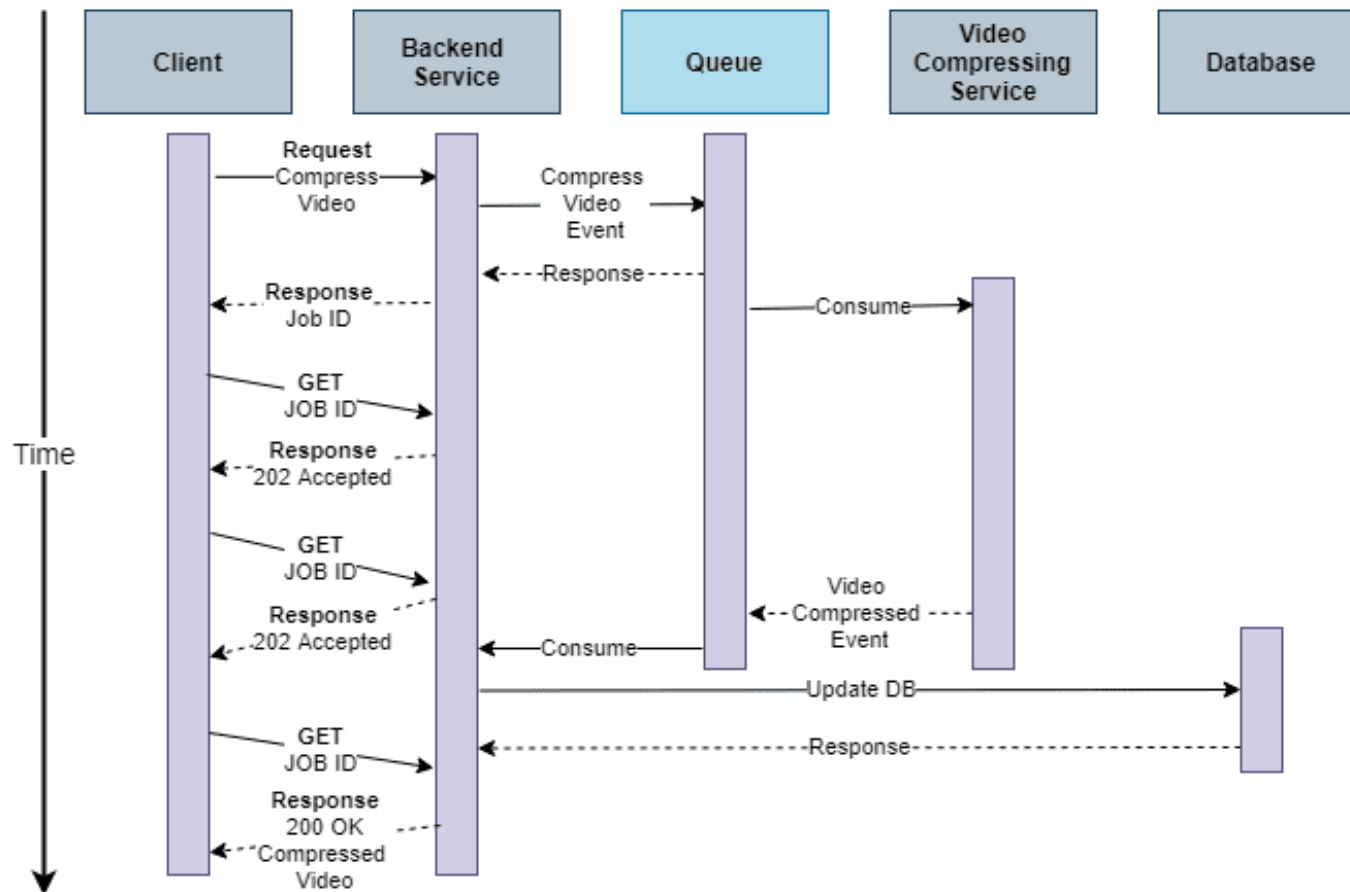


Performance IV

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot

Asincronía

- Invocar tareas sin esperar a completar
 - La recepción se confirma, pero no sabemos si se completó
- Permite a sistemas invocadores seguir operando
- Implementación
 - Manejo de errores y *timeouts* (*rejects*)
 - Monitoreo cuidadoso



Asincronía

- Nativo a estilos
 - EDA
 - Queues, Brokers
 - Implementaciones de SOA
 - Algunas implementaciones de Cli/Sv
 - *Promises, Callbacks, Async/await*
 - Reactive programming

Pros

- Sistemas liberados para tareas
- Elevación de la percepción de performance
- Mejor UX, DevX
- Implementación de concurrencia
- Hoy en día **mucha complejidad es oculta** (*Async/await*)



Cons

- Complejidad comprensiva
- *Callback hell*
- Manejo de errores complejo
 - A veces debe manejarse con patrones especiales
 - *Saga*
 - *Mediators*
- *Race conditions*
- Falta de sincronización
- Testing y debugging complejo



Asincronía: NodeJS

- Servidores Web altamente escalables
- Características notables
 - No multi-threads: Un sólo thread que maneja loop de eventos + 1 cola
 - Escrito en Javascript
 - Guiado por eventos, I/O asíncrono, no bloqueantes
 - Conexiones con alta concurrencia
 - JS everywhere
- Comunidad activa
 - NPM (¡ojo con seguridad!)
- Usado en la industria
 - Usualmente implementa las últimas novedades



NodeJS: Arquitectura

Javascript

C/C++

Librería estándar de Node

Bindings node
(socket, http, etc.)

V8

Pool de
threads
(libelo)

Loop de
eventos
(libdev)

DNS
(c-ares)

crypto
(OpenSSL)

Usos de NodeJS

- LinkedIn (Mobile Web App 20x faster)
- Pinterest
 - <http://highscalability.com/blog/2012/2/16/a-short-on-the-pinterest-stack-for-handling-3-million-users.html>
- Ebay (Data retrieval gateway)
- GitHub (downloads)
- Palm/HP (WebOS)
- Yahoo! Mail
- Rackspace
- Y muchos stacks más

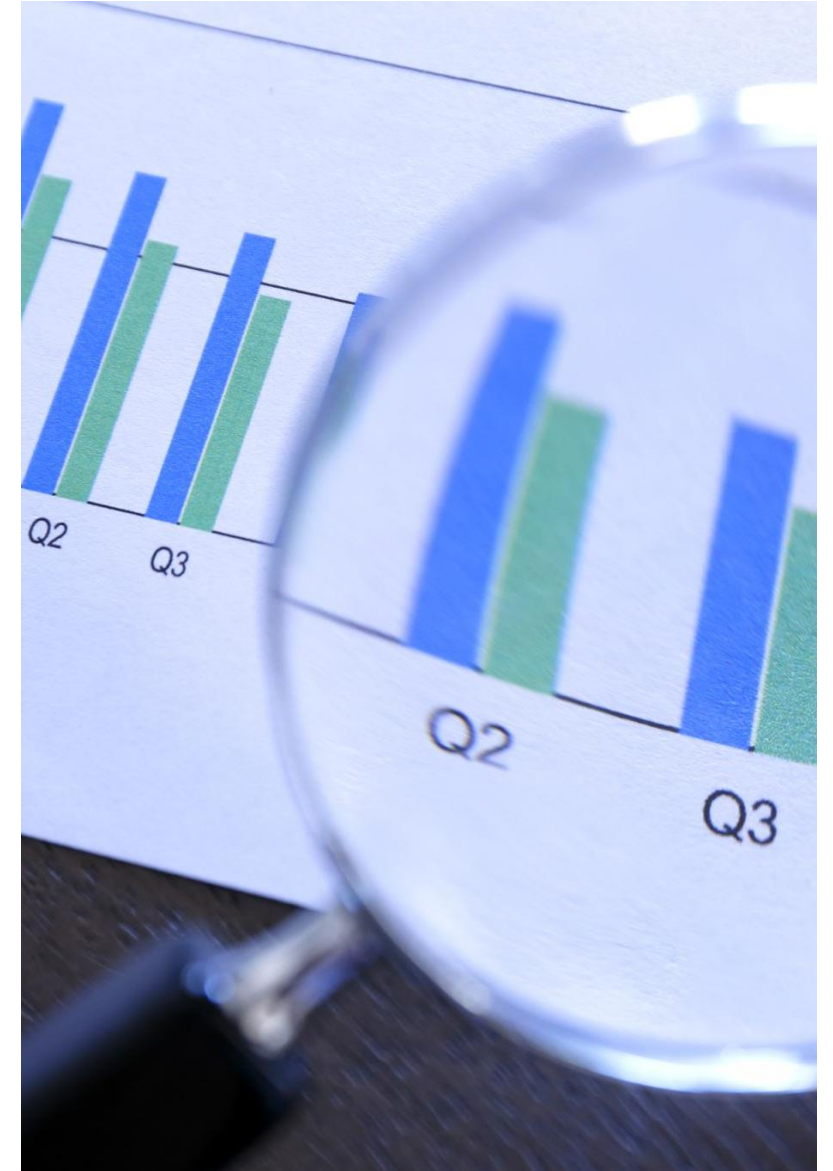
NodeJS: Desventajas

- No apto para
 - Sistemas de tiempo real estricto
 - Procesamiento de números
 - Datasets grandes en memoria (V8)
 - No es bueno con aplicaciones intensas en CPU
 - Se puede delegar a WebAssembly
- Soporte escaso a multithreading
- Requiere comprensión de async
- Single-thread puede volverse difícil

```
1 // Callback Hell
2
3
4 a(function (resultsFromA) {
5     b(resultsFromA, function (resultsFromB) {
6         c(resultsFromB, function (resultsFromC) {
7             d(resultsFromC, function (resultsFromD) {
8                 e(resultsFromD, function (resultsFromE) {
9                     f(resultsFromE, function (resultsFromF) {
10                         console.log(resultsFromF);
11                     })
12                 })
13             })
14         })
15     })
16 });
17
```

Sobre el performance

- Proceso constante de trabajo y medición
- Implemente las estrategias y tácticas óptimas para su caso
- No existe la bala de plata: solución menos mala





Performance

- IIC2173 - Arquitectura de sistemas de software
- Departamento de Ciencia de la Computación
- Escuela de Ingeniería – PUC
- Nicolás Acosta – Gerardo Crot